O'REILLY®

Introduction & Day 1 Overview

Ammar Mohanna

About Me

Dr. Ammar Mohanna

Academic Background

- Ph.D. in Edge Artificial Intelligence
- Master's in Software Engineering

Professional Roles

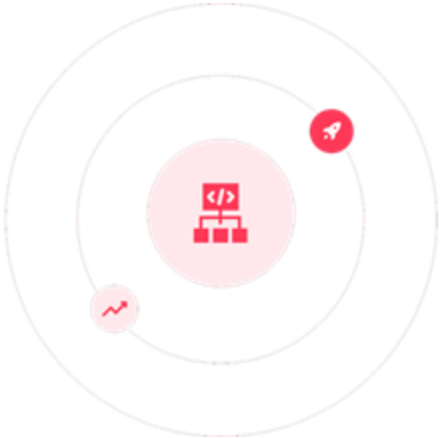
- Lecturer at the American University of Beirut (AUB)
- Senior AI Consultant at EDT&Partners

Let’s connect!

Dr. Ammar Mohanna

Why MLOps

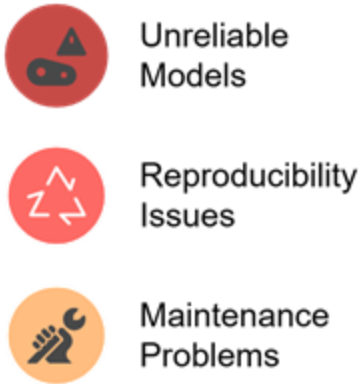
- Machine learning is **transforming** industries
- But building a model isn't enough—**deploying, managing,** and **scaling** models in production is the real challenge



What is MLOps?

- Research → Production

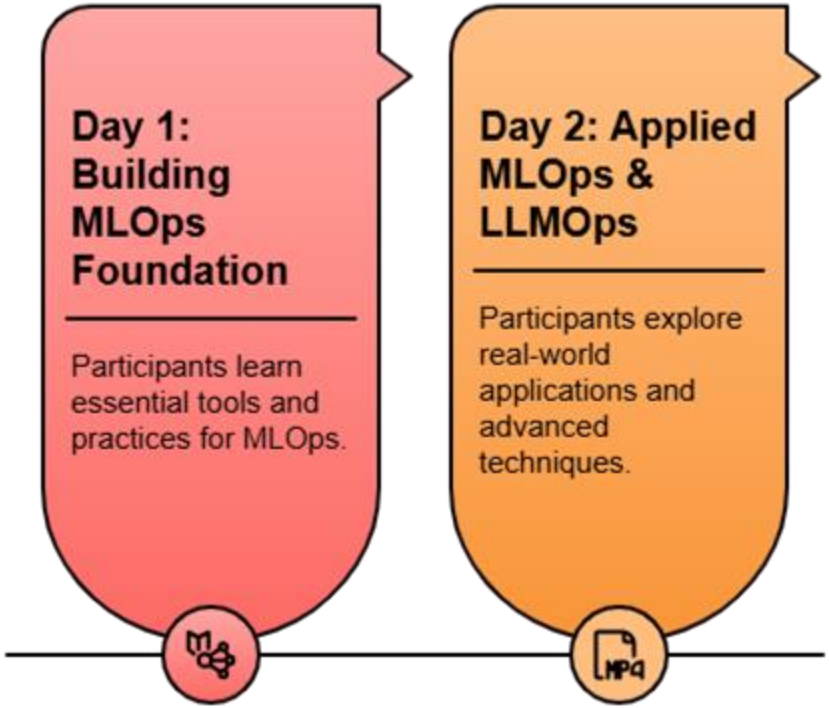
- Without MLOps, models become:



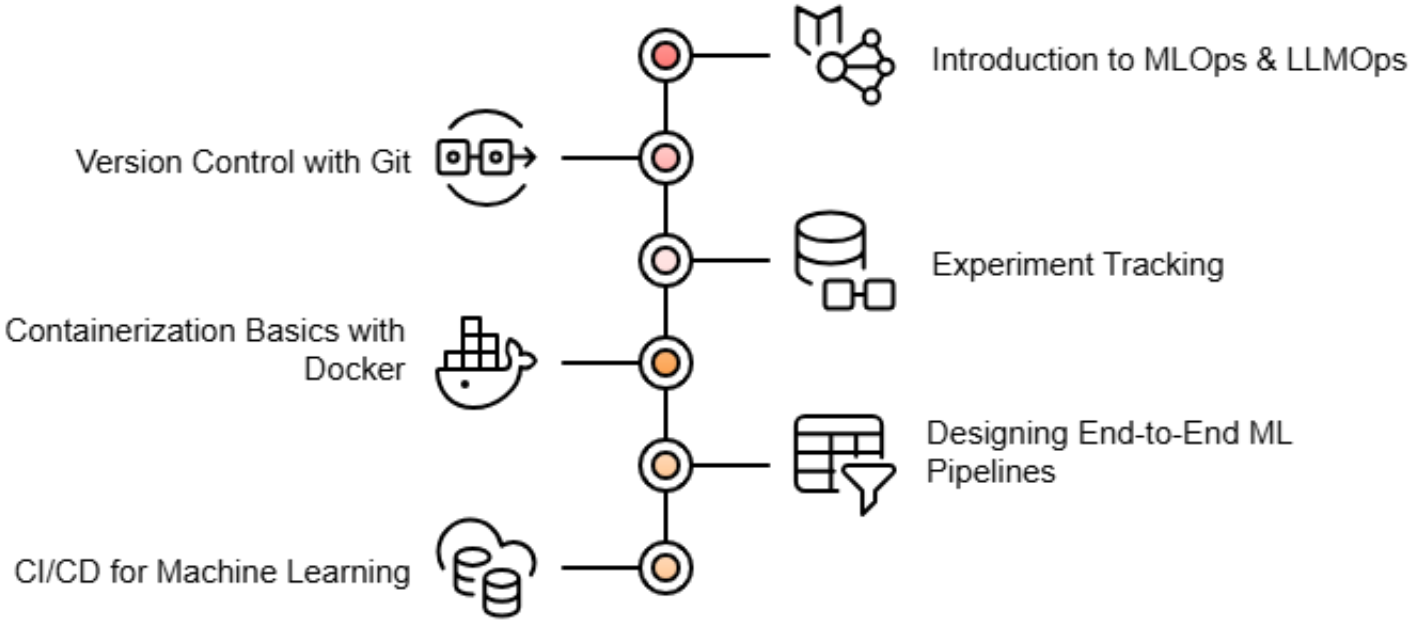
Bootcamp Objective

- Bridge the **gap** between **developing** ML and LLM models and **deploying** them at scale
- Equip you with the **essential skills** to:
  - Streamline ML workflows
  - Ensure reproducibility
  - Optimize performance

Bootcamp Overview



Day 1


Introduction to MLOps & LLMOps
Version Control with Git
Experiment Tracking
Containerization Basics with Docker
Designing End-to-End ML Pipelines
CI/CD for Machine Learning

Let’s Get Started


O'REILLY®

O'REILLY

Introduction to MLOps & LLMOps

Ammar Mohanna

Learning Objectives

By the end of this lesson, you will be able to:

- Understand the **ML lifecycle** in production
- Recognize **challenges** and the role of **MLOps**
- Understand what is **Large Language Model Operations (LLMOps)**
- Differentiate between **LLMOps** and **MLOps**
- Identify **challenges** in deploying and maintaining LLMs

ML Lifecycle

Understanding ML in Production

- The algorithm is only a **small** part of an ML system in production

ML System Requirements

Maintainability

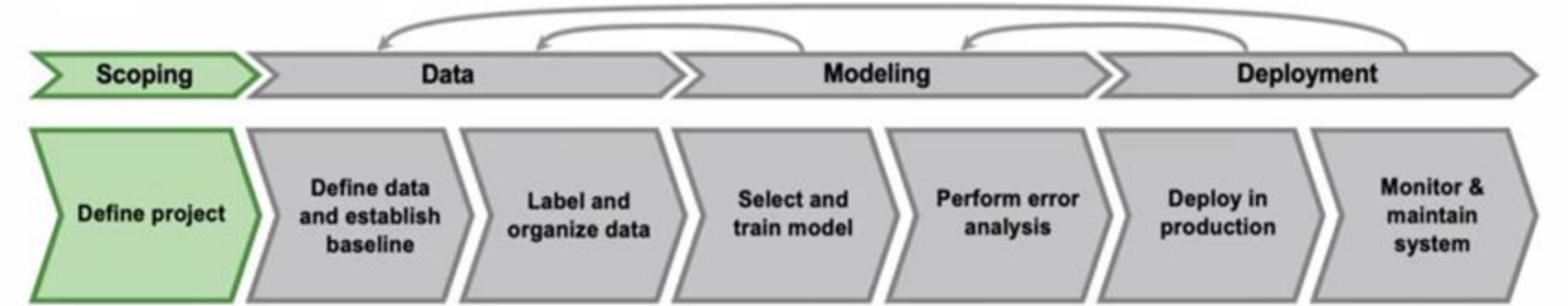
Reliability

Adaptability

Scalability

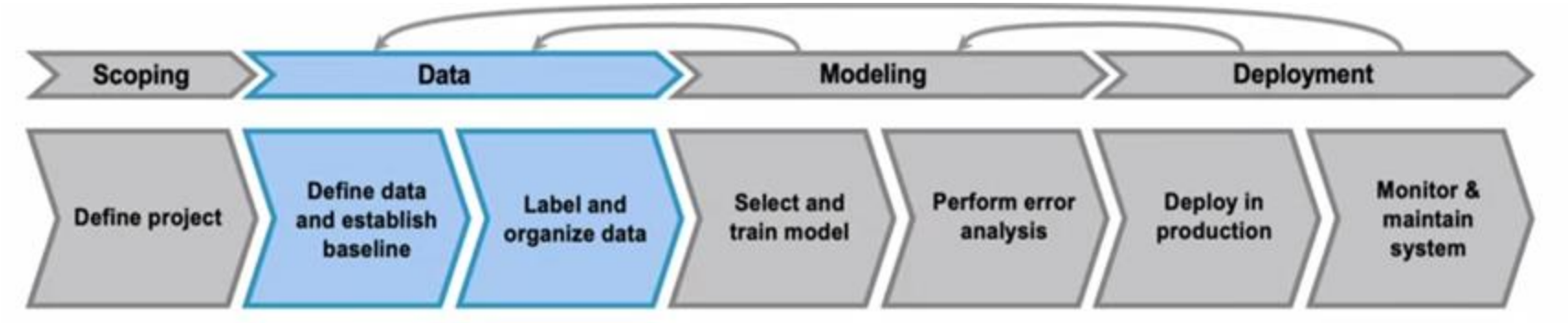
Scoping

- Decide on the **project** to work on
- Define the **goals, stakeholders, resources,** and **constraints**



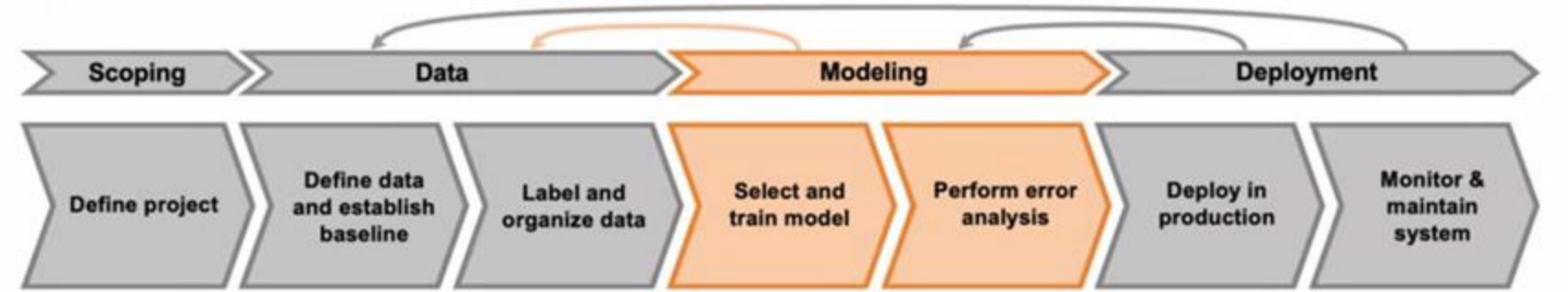
Data

- **Gather**, **clean**, and **preprocess** data for training
- Is the data **labelled consistently?**



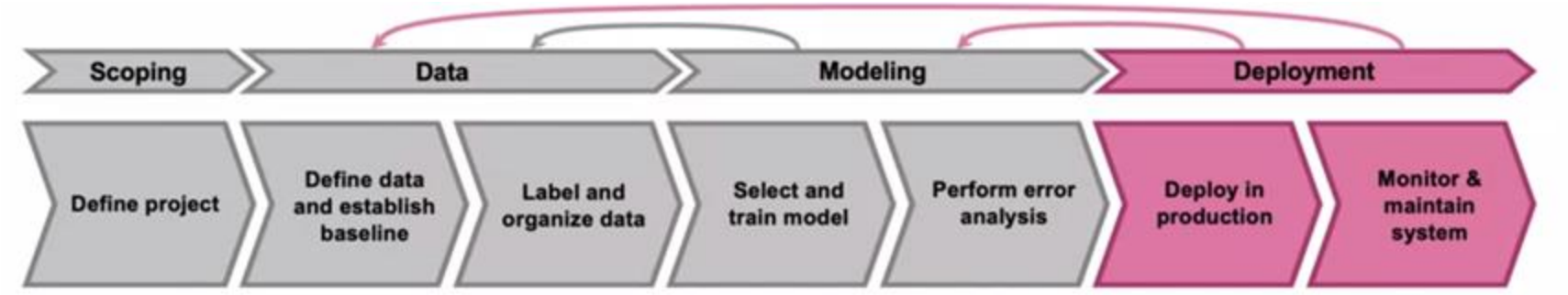
Modelling

- Optimize **code (model)**
- Optimize **hyperparameters**
- **Error Analysis:** identify weaknesses by examining incorrect predictions



Deployment

- Make the model **accessible** to users
- Track **concept drift** and **adapt** to changes



Example: Speech Recognition

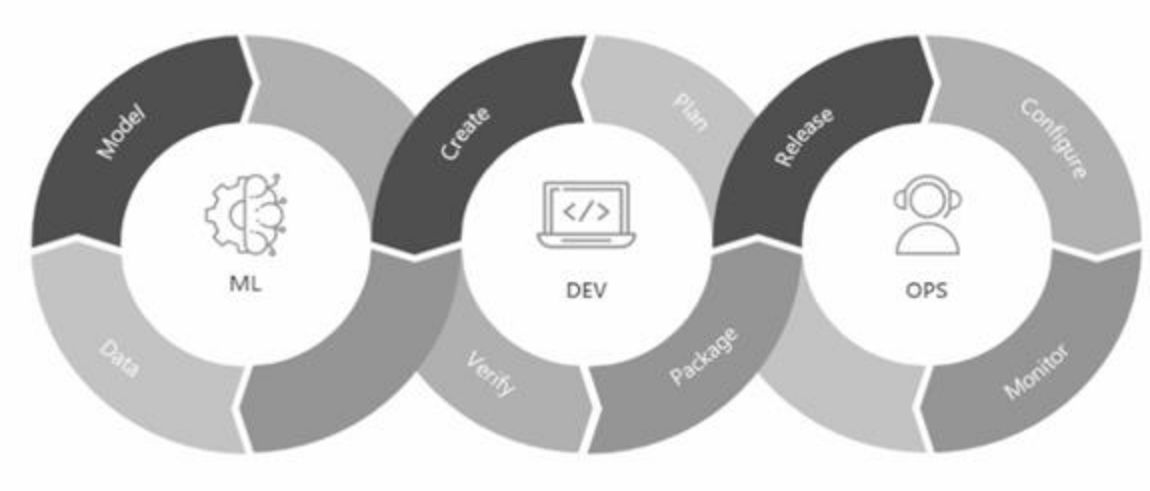
MLOps in Production

Challenges in ML Production

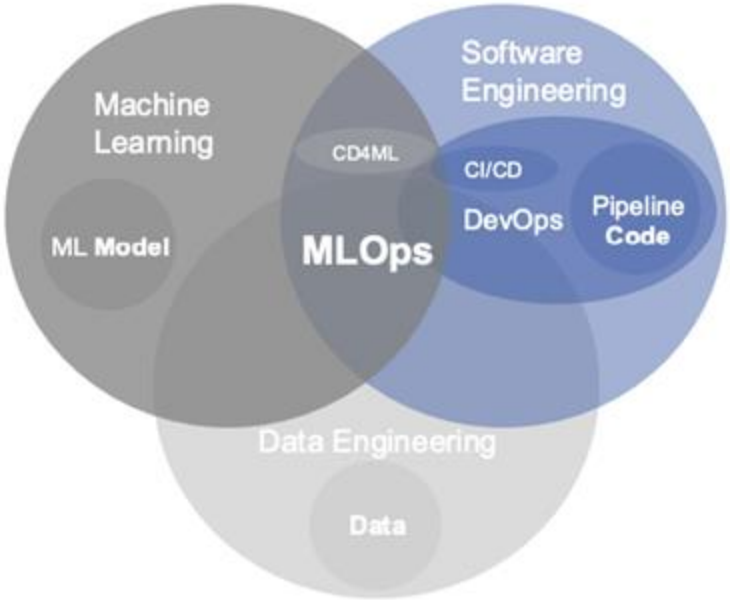
- **Testing** and **versioning** both code and data
- **Scaling** to handle large models
- Ensuring **fast response times** (e.g., autocomplete suggestions)
- **Monitoring** and **debugging** deployed models

What is MLOps

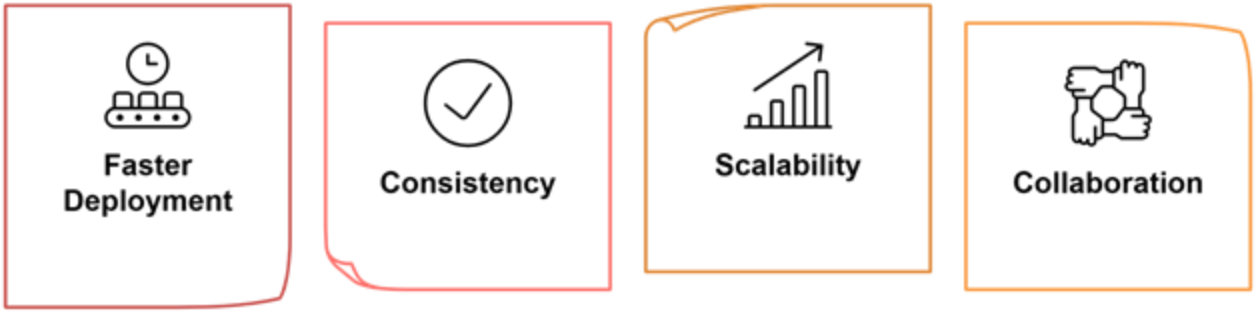
ML → Production



Where MLOps Lies



Benefits of MLOps



Key Considerations When Moving to Production

- Different **stakeholders** have different requirements
- Fast **inference**, low **latency**
- **Data drift:** real-world data constantly shifts over time

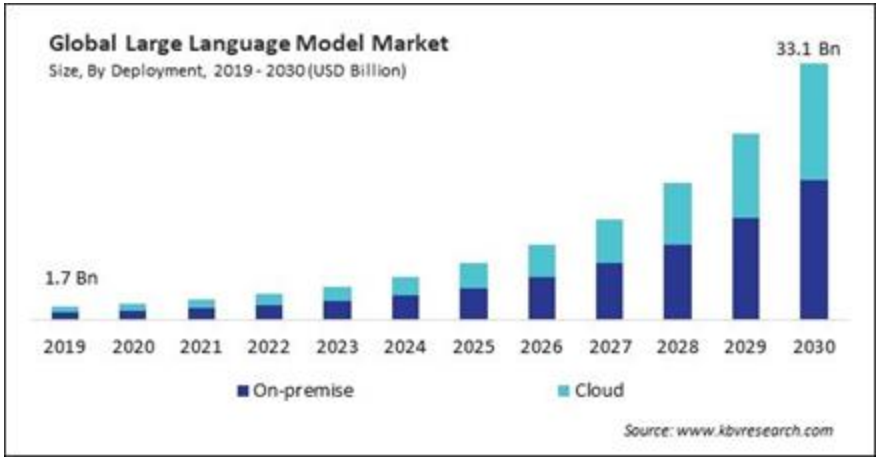
From MLOps to LLMOps

LLMs

- Specialized foundation models trained for **language-based tasks**
- Predict and generate **human-like text**
- Power **NLP tasks**

Rise of LLMs

- Enterprises increasingly **adopt LLMs** for value and profit
- **Rising demand** drives LLMOps adoption



Challenges of Building with LLMs

Size and Complexity

LLMs have billions of parameters, making training and evaluation complex.

Training Scale and Duration

LLM training needs vast datasets and high resources.

Ethical Considerations

Bias in training data can affect responses.

Costs

Training and maintaining LLMs is costly, especially inference.

What is LLMOps

- **LLMOps (Large Language Model Operations)** refers to practices, tools, and workflows for **deploying, managing,** and **maintaining** LLMs in production

Four Core Goals of LLMOps

Ensure Safety

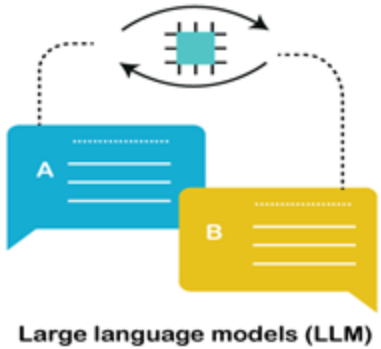
Achieve Scalability

Maintain Robustness

Enhance Reliability

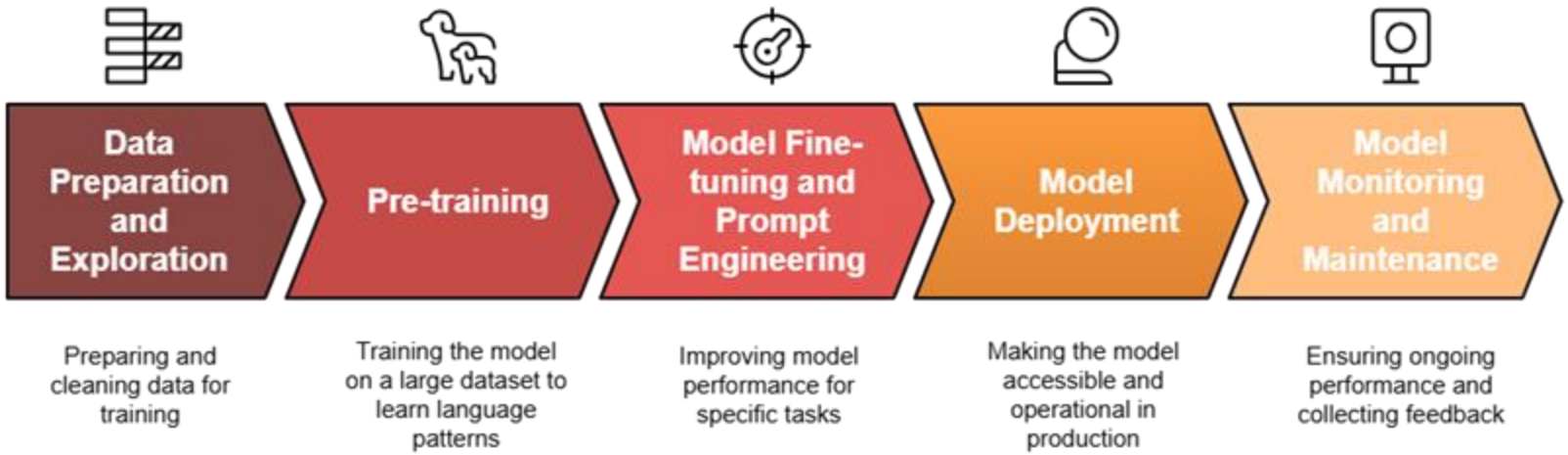
Need for LLMOps

- Ensuring LLMs remain **efficient** and **reliable** in production is challenging
- LLMOps optimizes **performance, scalability,** and **resource management** for large models



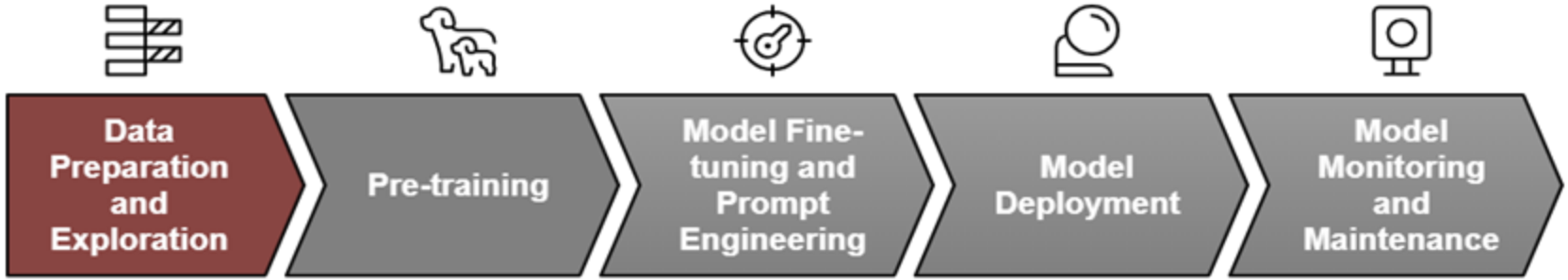
LLMOps Workflow

LLMOps Workflow



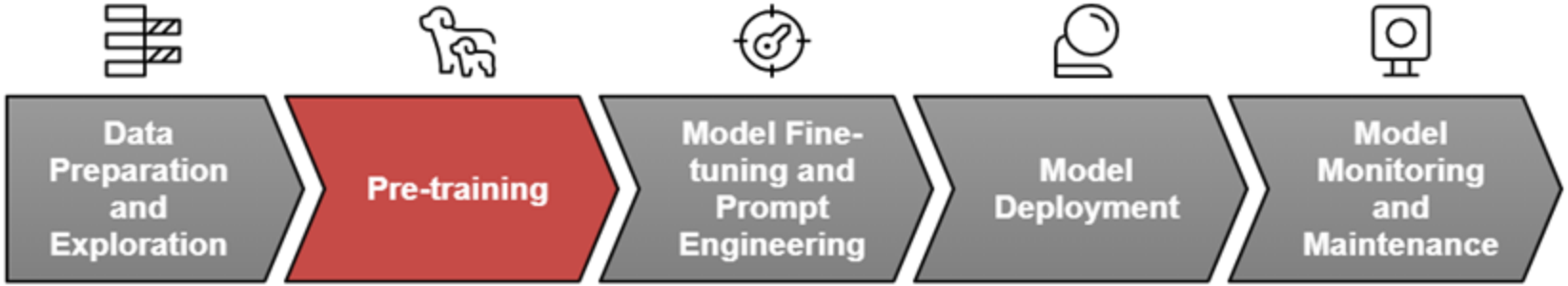
Data Preparation and Exploration

- Cleaning, Exploring, and Preparing the data
- **High-quality** data



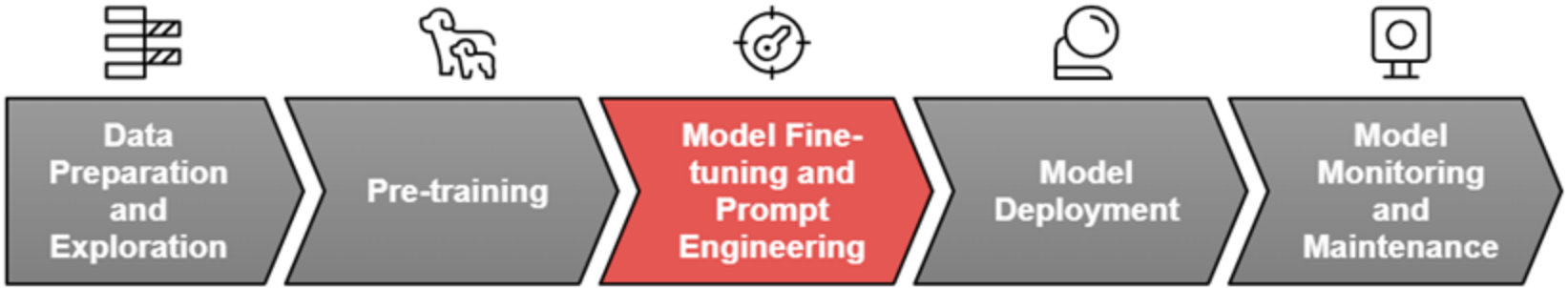
Pre-training

- Often uses **pre-trained models** on large text corpora
- Trains the model to learn **general language patterns**



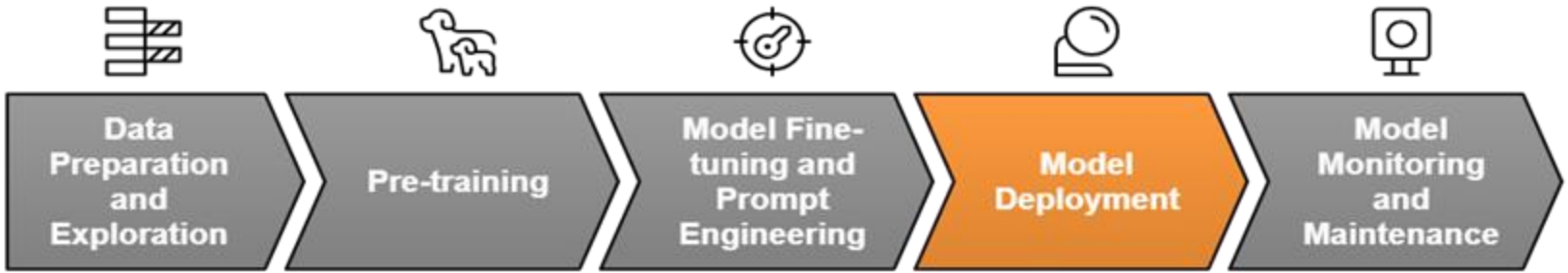
Model Fine-tuning and Prompt Engineering

- **Fine-tunes** the **pre-trained** model
- Optimizing prompts



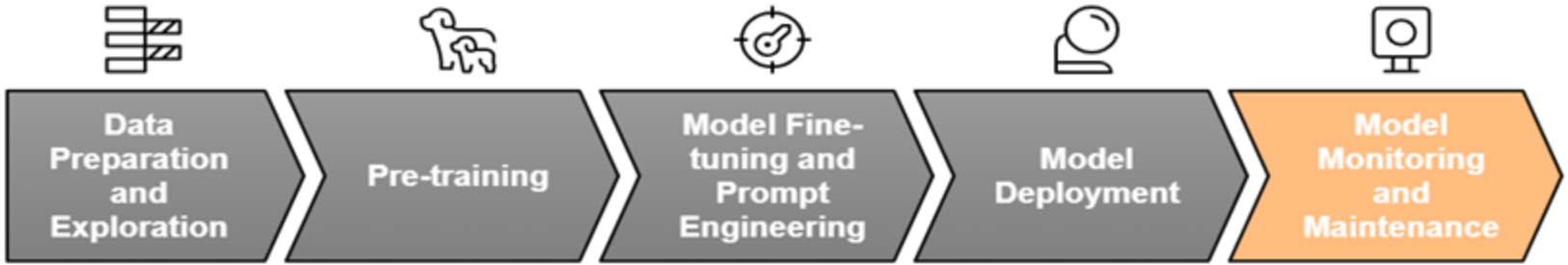
Model Deployment

- Makes the model **accessible**
- **Integrating** the model



Model Monitoring and Maintenance

- **Ongoing** monitoring
- Collects **user feedback**



MLOps vs LLMOps

From MLOps to LLMOps

- **MLOps** focuses on traditional ML models (classification, regression, recommendation)
- **LLMOps** extends MLOps to generative models, requiring **new strategies** for:
  - Fine-tuning & prompt engineering
  - Scalable deployment & optimization
  - Real-time monitoring

MLOps vs. LLMOps

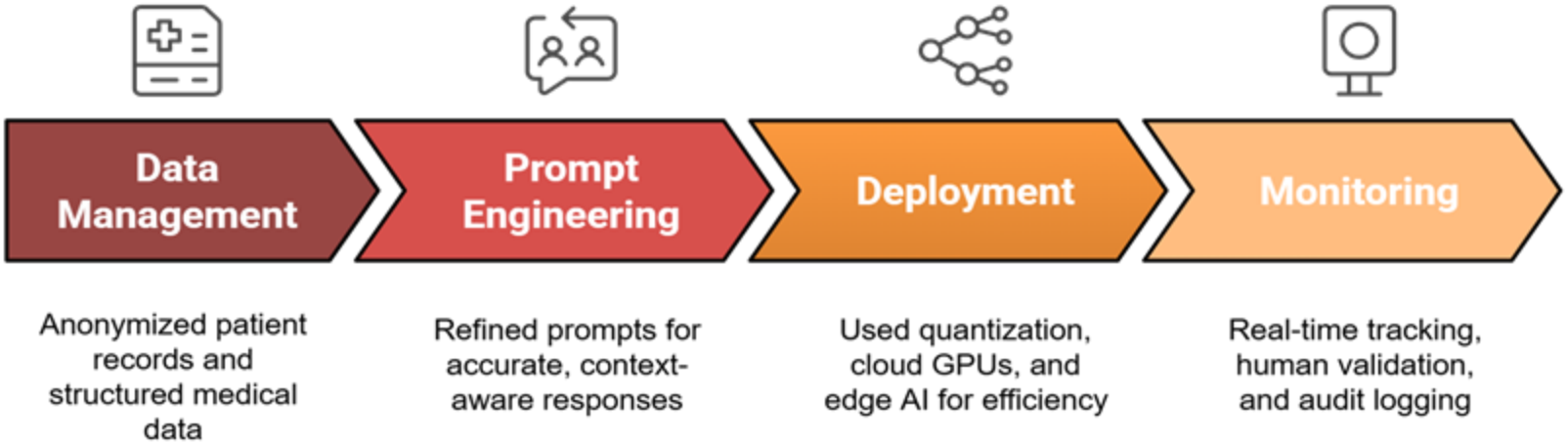
Aspect	MLOps	LLMOps
Data Collection	Simple	Requires Filtering and Diversity
Preprocessing	Fixing missing data, outliers	Handling bias, toxicity
Model Size	≤ 200M params, manageable	100B+ params, high compute need
Training Time	Short, single-node setups	Weeks/months, distributed GPUs/TPUs
Evaluation	Well-defined metrics	Unbounded, unpredictable
Model Behavior	Static in production	Dynamic, evolves with use

Case Study: LLMOps in Healthcare

Medical Diagnosis Support

- A healthcare provider aimed to deploy an AI-driven diagnosis assistant but faced key challenges:
  - Ensuring **accuracy & reliability**
  - Maintaining **data privacy & compliance**
  - Enabling real-time inference with **minimal latency**
  - Preventing **hallucinations** for trustworthy responses

LLMOps for Medical Diagnosis Support



Outcomes

Diagnostic Precision

Improved Efficiency

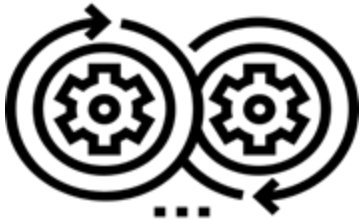
Scalable System

Regulatory Compliance

Summary

Key Takeaways

- **ML lifecycle**: scoping, data, modeling, deployment, managed by **MLOps**
- Focus on **inference** and **scalability**
- **LLMOps** addresses model size, compute, and behavior challenges
- **LLMOps** is essential for large language model deployment



Discussion Question

What challenges have you faced (or do you anticipate) in deploying ML models?


O'REILLY®

O'REILLY

Introduction to Git for ML Workflows

Ammar Mohanna

Learning Objectives

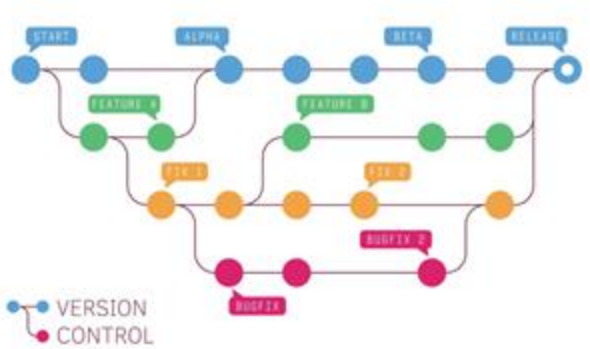
By the end of this lesson, you will be able to:

- Understand **version control** and its role in managing code changes
- Explain **Git's importance** in ML workflows for collaboration and reproducibility
- Use **Git** to track code, collaborate, and manage model versions
- Apply **best practices** for organizing ML projects

Version Control

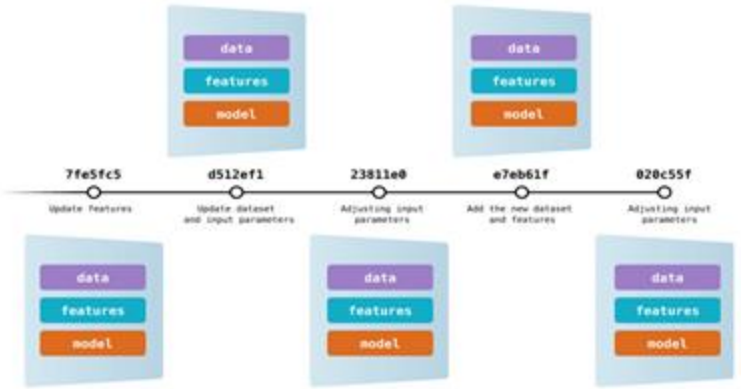
Version Control

- Tracks **changes** to files over time
- Allows **reverting** to **previous** versions if needed
- Commonly used in **software development**



Version Control in ML

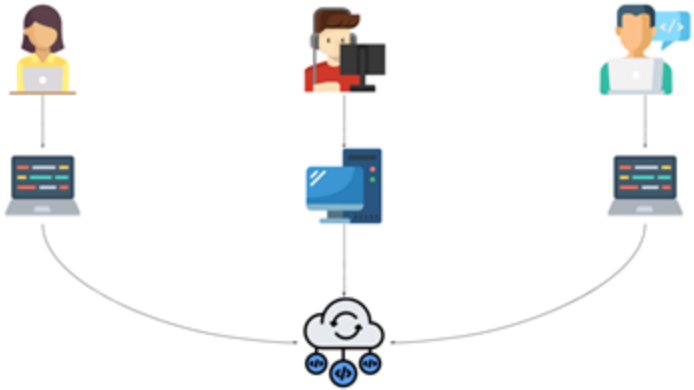
- Enables **iterative** ML experimentation
- Ensures **reproducibility** of **code** & **data**
- Allows **rollback** for errors & model issues



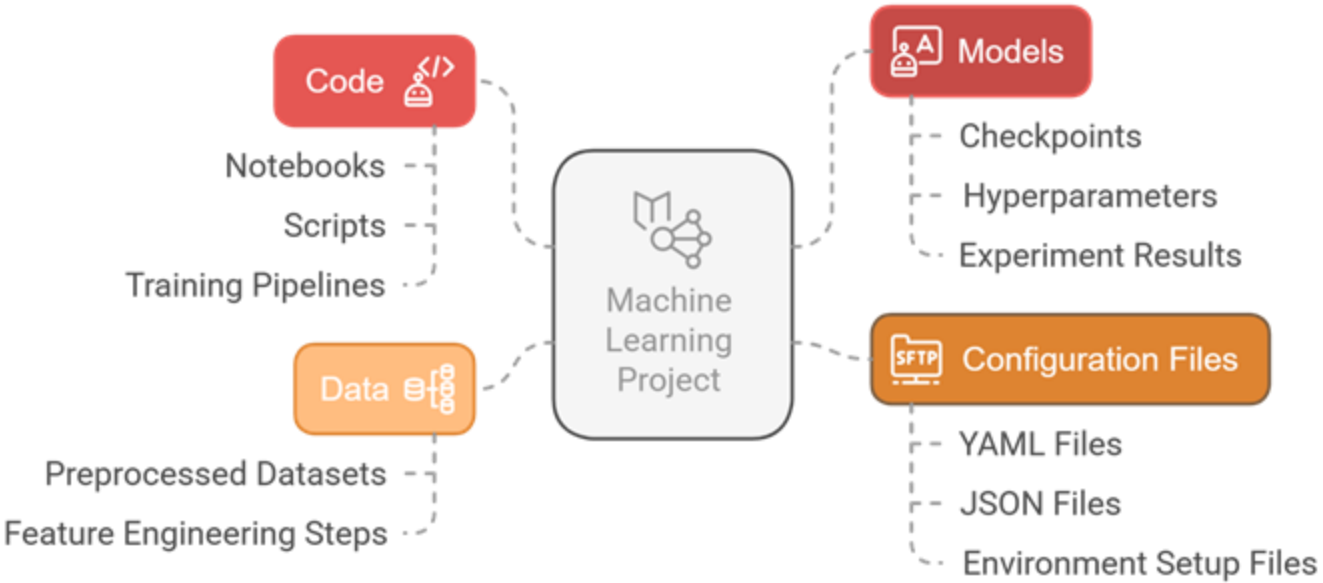
Git & Github

Git

- **Git:** Widely used for tracking file changes
- **Use:** Collaboratively developing source code



Tracking ML Code, Data, and Models



Github

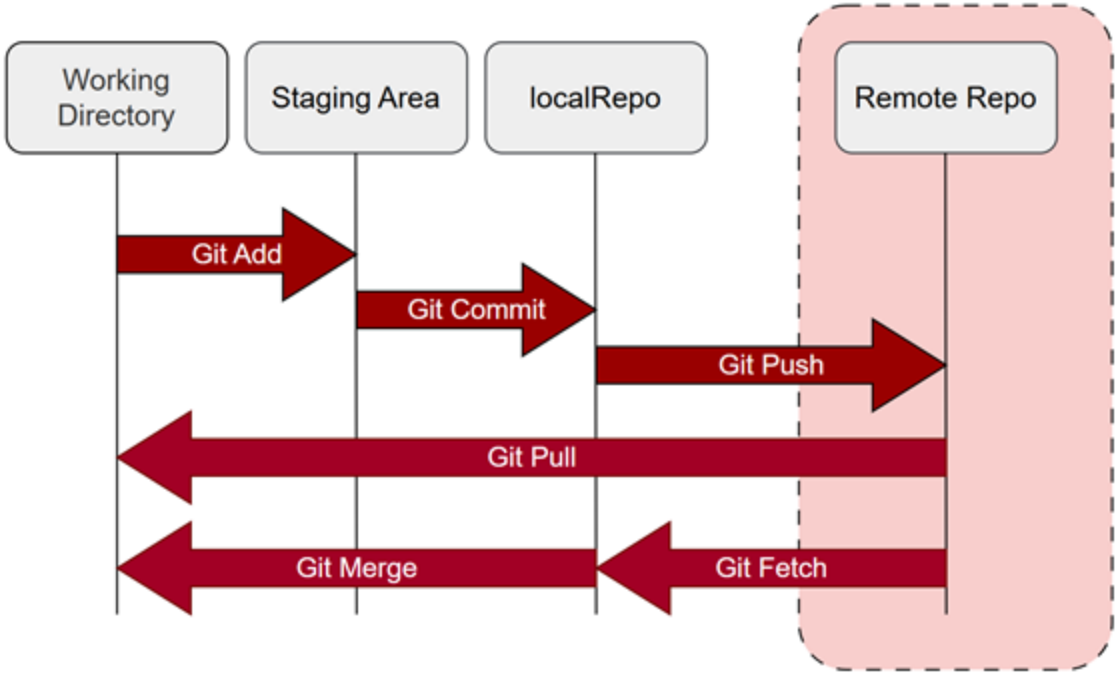
- **Cloud-based platform** for hosting and managing Git repositories
- Built on **top** of **Git** for version control and collaboration

Why Github


Distribute Software
Easily share software with users.
Report Issues
Notify about problems or bugs in the software.
Request Features
Suggest new functionalities for the software.
Collaborate
Work together through pull requests for improvements.

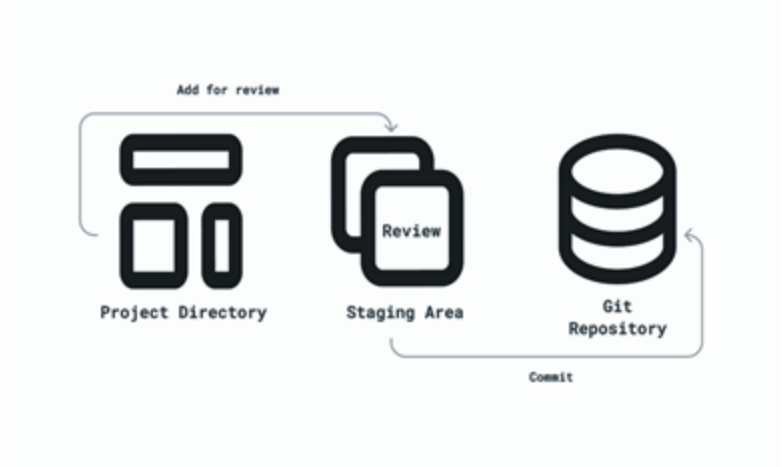
Git Workflow

Git Workflow



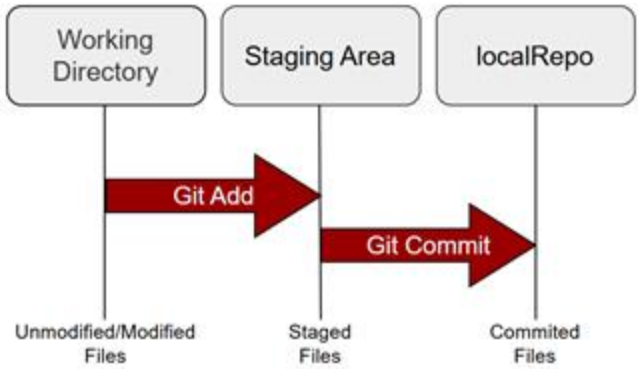
Git Workflow

- **Working Directory**: where you make changes to files
- **Staging Area**: prepares changes for the next commit
- **Local Repository:** stores committed changes permanently



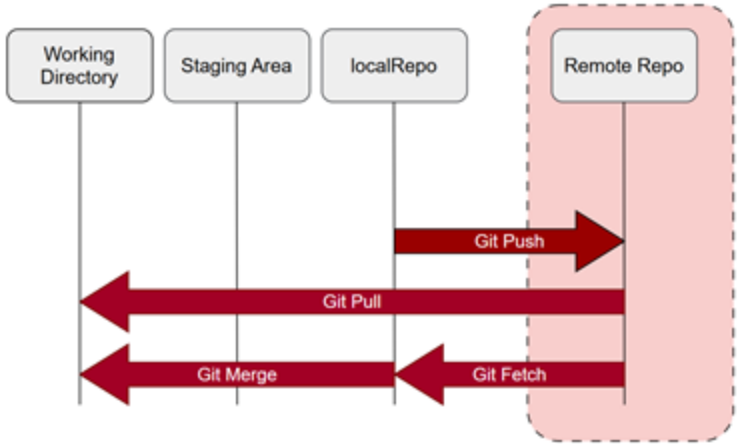
Basic Git Workflow

1. Modify files in the **working directory**
2. **Stage** changes (git add)
3. **Commit** snapshots (git commit -m "message")



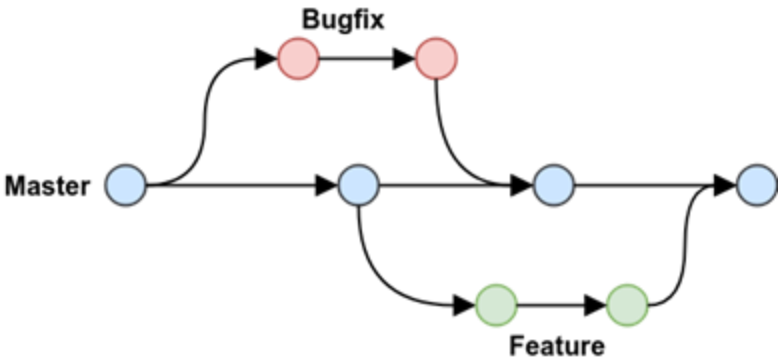
Interaction with Remote Repo

- **Pull:** Fetches updates from the remote and integrates them into your local branch
- **Push:** Sends your local commits to the remote repository



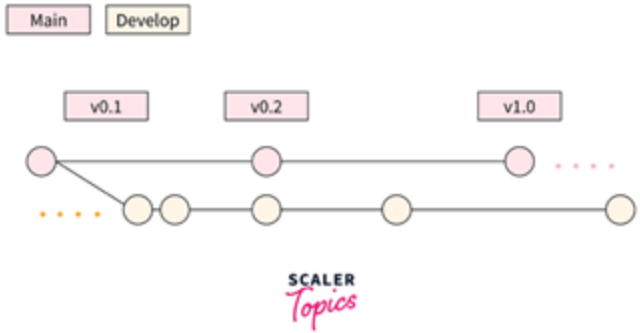
Git Branch

- **Independent** development line for features, fixes, or experiments
- Does **not affect** the main project

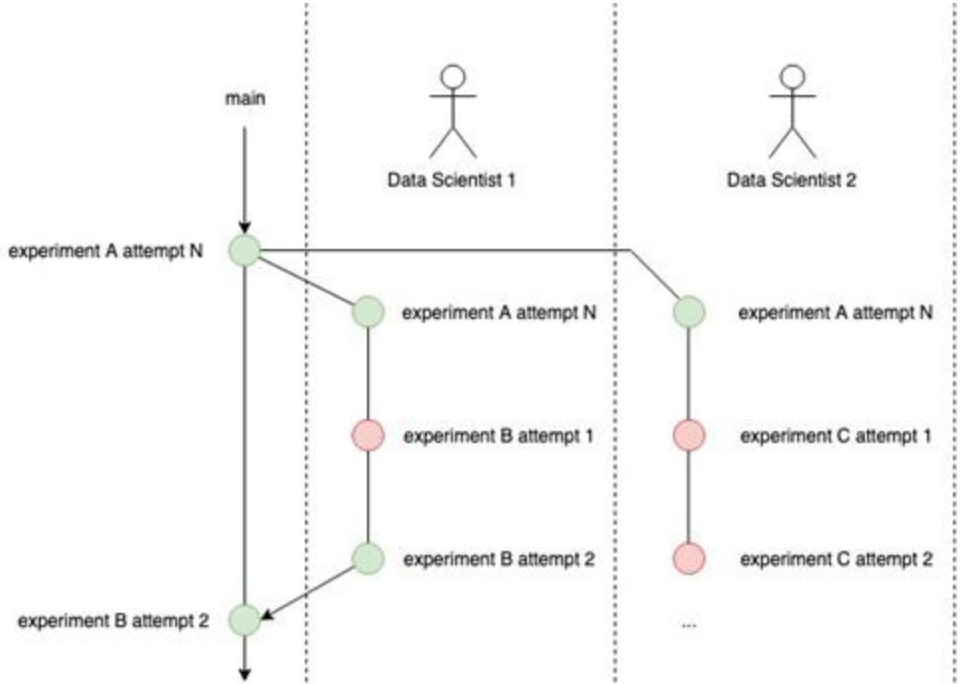


Git Branching for ML Experimentation

- Create branches for **different** experiments
- Compare **model versions** before merging
- Keep main branch **stable** for production-ready code



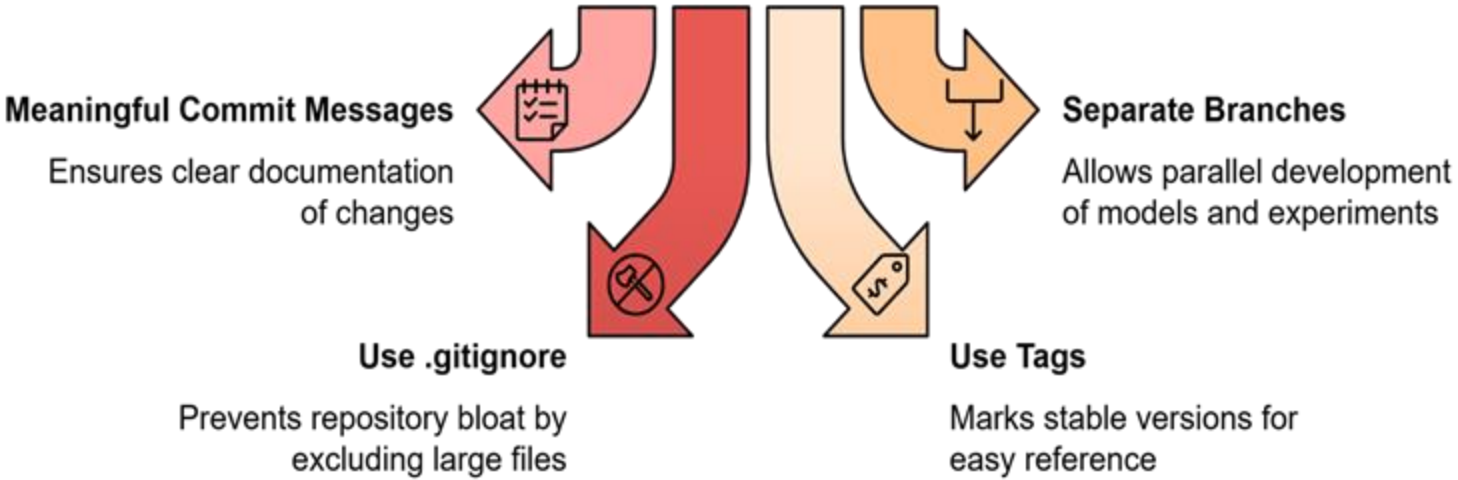
Example


main
Data Scientist 1
Data Scientist 2
experiment A attempt N
experiment A attempt N
experiment A attempt N
experiment B attempt 1
experiment C attempt 1
experiment B attempt 2
experiment C attempt 2
experiment B attempt 2
...

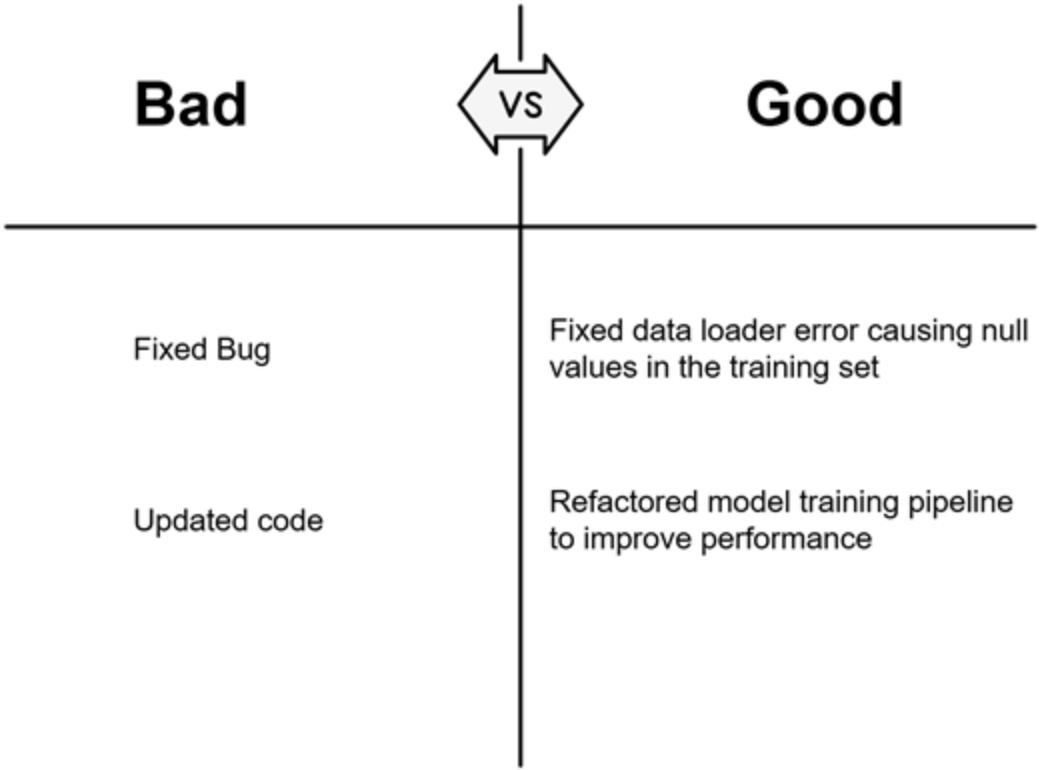
Handling Data in Git

- Use **.gitignore**
- Track **data preprocessing scripts**
- Use **Git LFS** or **DVC** for large dataset versioning

Best Practices for Git in ML Projects



Example


Bad
VS
Good
Fixed Bug
Fixed data loader error causing null values in the training set
Updated code
Refactored model training pipeline to improve performance

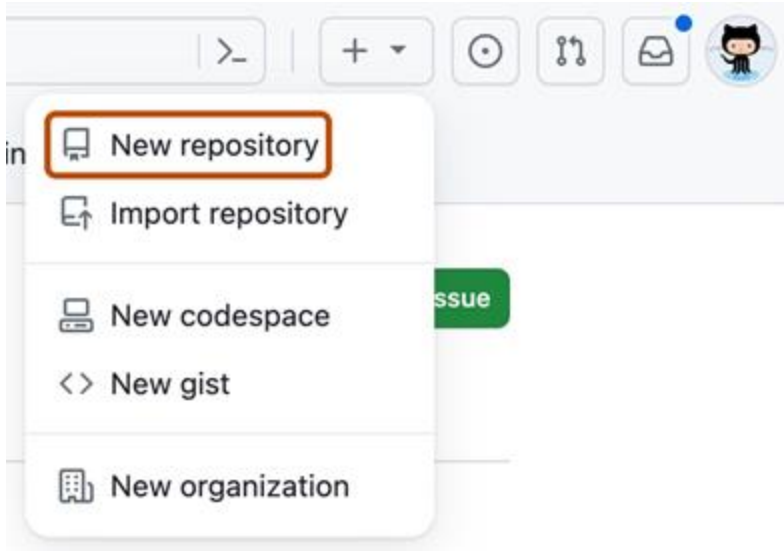
Hands-on git

Install git

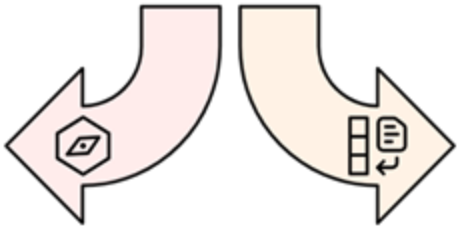
- **Download Git :** Visit git-scm.com and download the latest version for your OS
- **Verify Installation:** git --version

Install git

- **Sign up** at github.com
- Create a **repository** to store your code



Setting up a Git Repository



New Local Repository

Use this to start a new project in a local directory.

git init

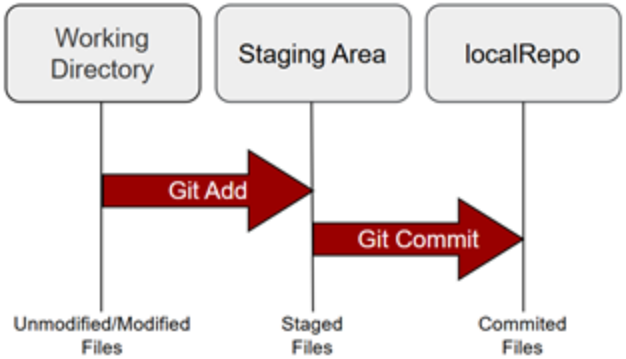
Clone Remote Repository

Use this to work on an existing project from a remote source.

git clone “url” “dirname”

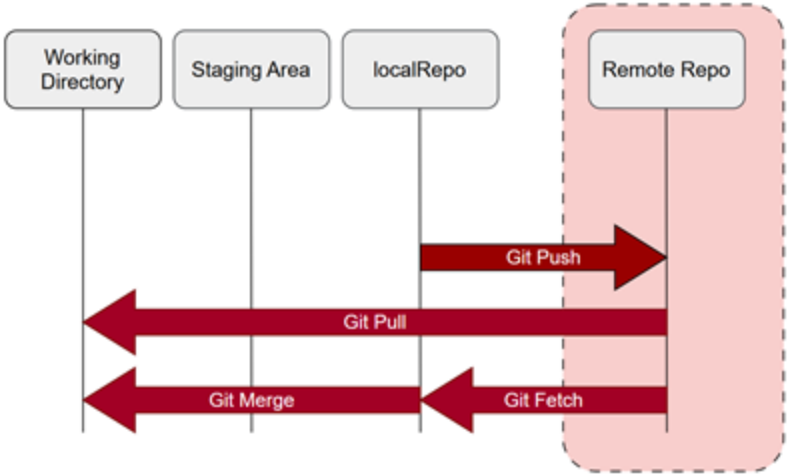
Making and Tracking Changes

1. **Create** or **modify** a file, then check its **status** (git status)
2. **Stage** changes (git add)
3. **Commit** changes (git commit -m "message")

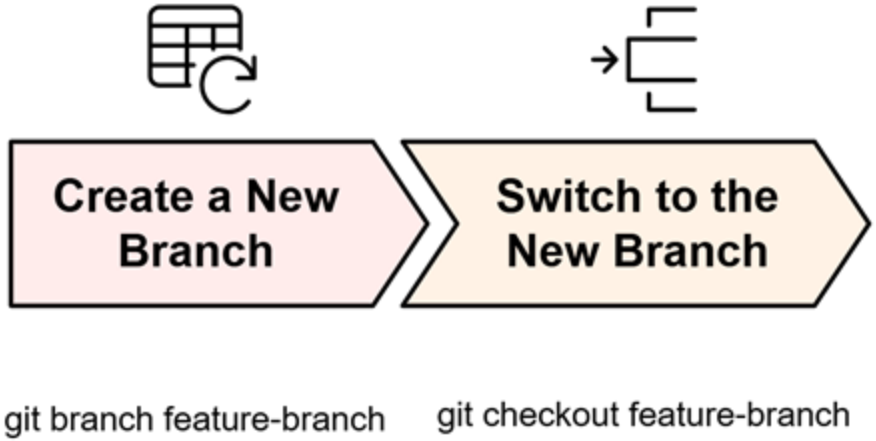


Collaborating with Remote Repositories

- **Sync changes from remote:** Use **git pull origin main** or **git fetch** followed by **git merge**
- **Push local commits:** Use **git push origin main** to update the remote repository

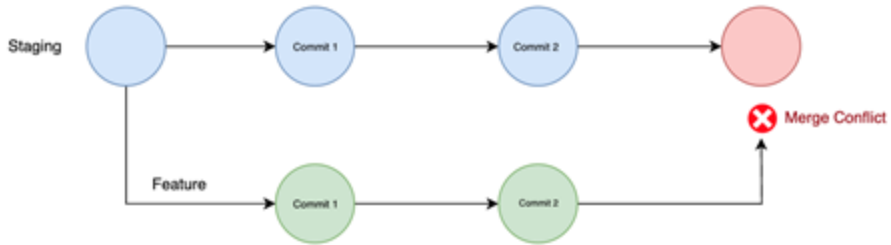


Git Branching



Merge Conflict

- Occurs when Git **cannot automatically merge** changes from different branches
- Happens when multiple contributors **modify** the **same** file or line



Using Fork Tool

- A **graphical interface** for **Git** that simplifies version control
- **Automates processes** like branching, merging, and conflict resolution with a few clicks
- **Download and install Fork:** https://git-fork.com/

Git Commands

Command	Description
git clone url [dir]	Copy a Git repository so you can add to it
git add files	Add file contents to the staging area
git commit	Record a snapshot of the staging area
git status	View the status of your files in the working directory and staging area
git diff	Show the difference between what is staged and what is modified but unstaged
git help [command]	Get help information about a particular command
git pull	Fetch from a remote repository and merge into the current branch
git push	Push your new branches and data to a remote repository

Summary

Key Takeaways

- **Version control** tracks changes and enables collaboration
- **Git** for version control, **GitHub** for hosting
- **Git Workflow:** Modify, stage, commit; use branches for experiments
- **Best Practices:** Meaningful commits, clean repos, track releases with tags

Poll: Which version control tools are you currently using?

- Git (e.g., GitHub, GitLab, Bitbucket)
- Subversion (SVN)
- Mercurial
- None
- Other (please comment)


O'REILLY®

Break

O'REILLY

Experiment Tracking

Ammar Mohanna

Learning Objectives

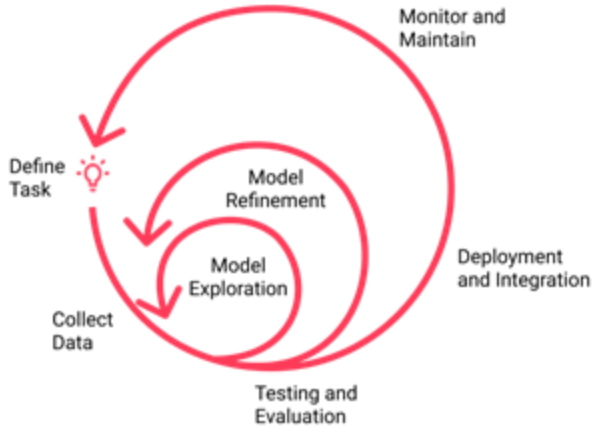
By the end of this lesson, you will be able to:

- Grasp the importance of **experiment tracking**
- Apply **best practices** for structured and scalable projects
- Discover the **capabilities** of **MLflow**
- Learn the **key elements** to track when experimenting with LLMs

Experiment Tracking

Iterative Model Development

- Continuously **refine** and **compare** models to ensure improved performance and suitability for **production**



Challenges Faced in Production

Reproducibility Issues

Ensuring models can be retrained and results replicated

Traceability

Tracking changes and identifying versions linked to outcomes

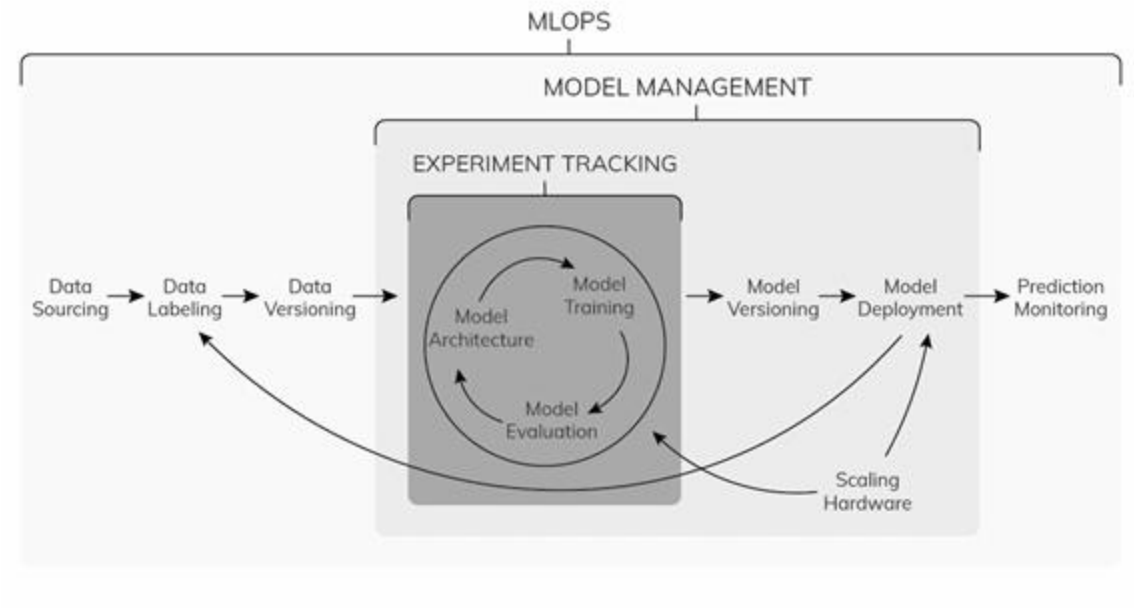
Deployment Inconsistencies

Preventing the deployment of incorrect or outdated models

Model Drift and Staleness

Monitoring performance and updating models as needed

Model Management and Experiment Tracking

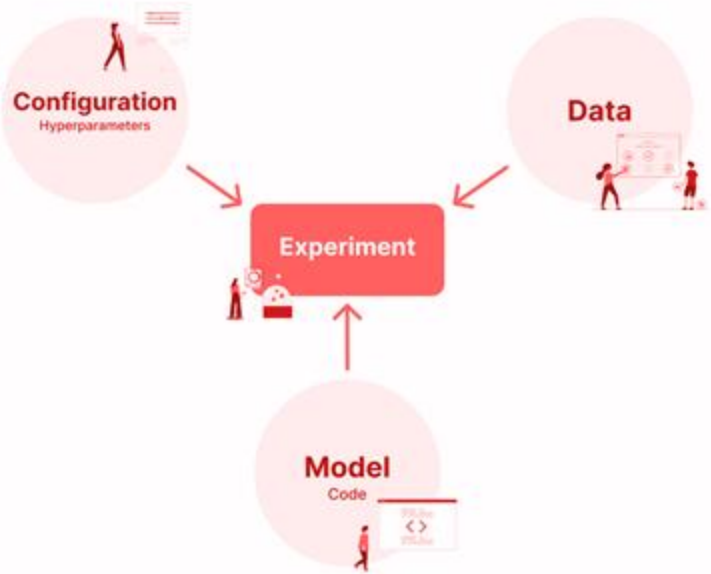


Experiment Tracking

- **Logs** and **organizes** machine learning experiments
- Manages **experiment iterations** efficiently
- **Refine** and **compare** all aspects of the experimentation process

Experiment Tracking

What to Track



Tools for Experiment Tracking

Best Practices

Track Everything That Matters

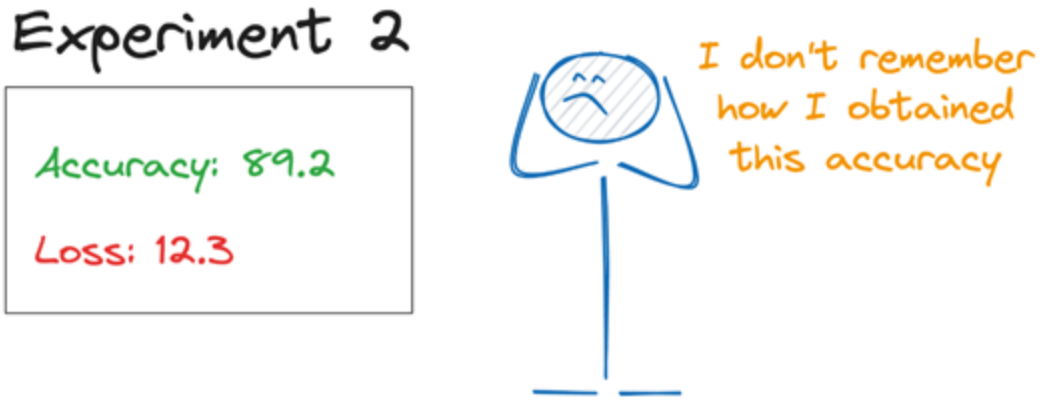
Use a Dedicated Experiment Tracking Tool

Ensure Reproducibility

Standardize Naming & Tagging

Overview of MLflow

Traditional Model Management Challenges



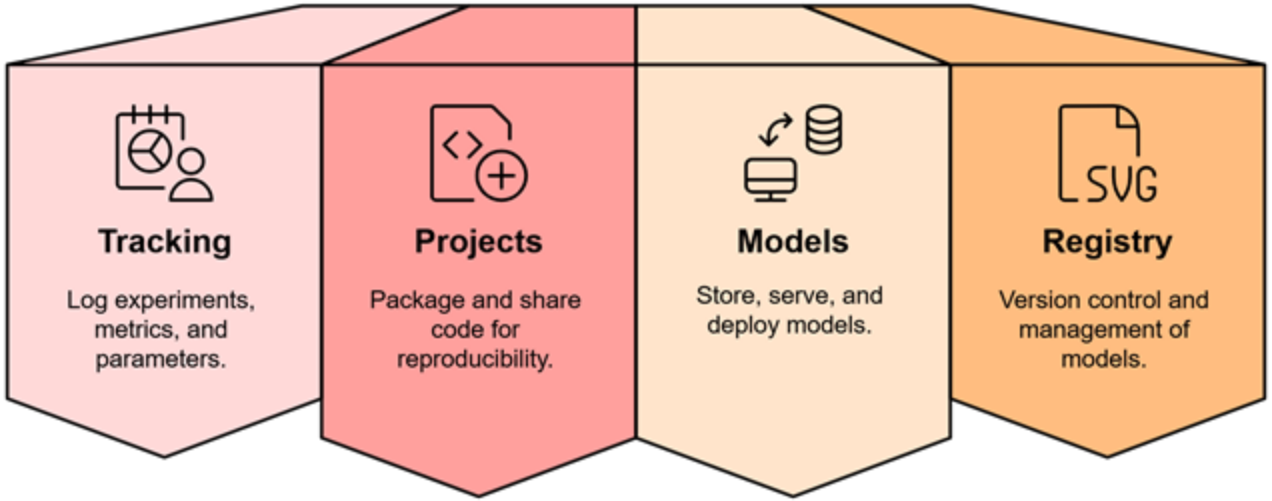
Improved Model Management with MLflow

Introduction to MLflow

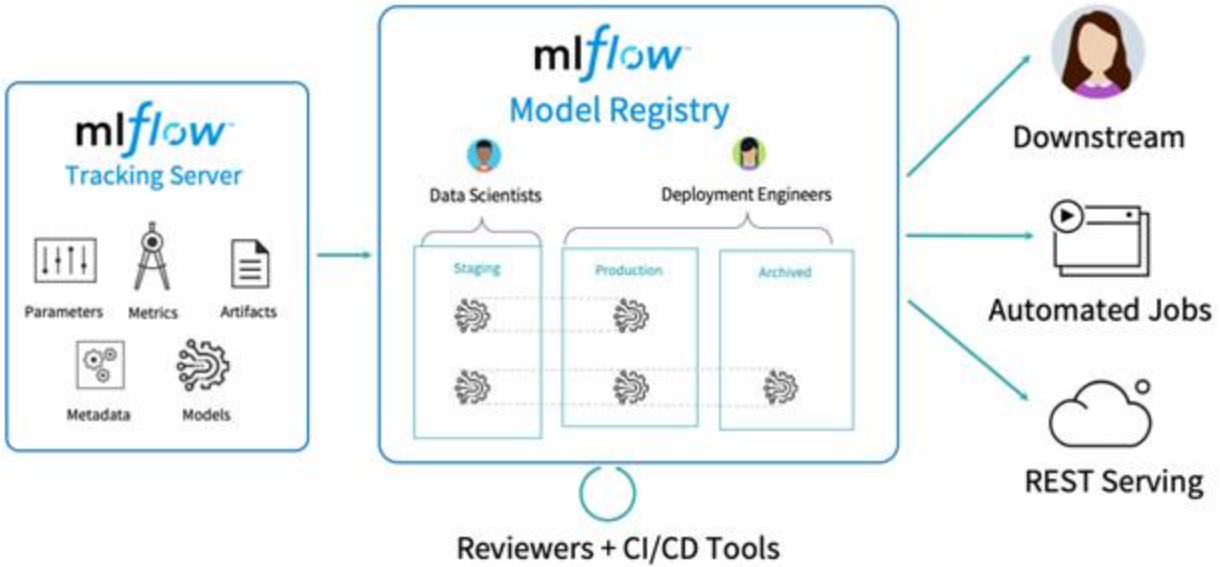
- An **open-source platform** for managing the ML lifecycle.
- Provides tools for **experiment tracking, model versioning, and deployment**

Why Use MLflow?

Key Components of MLflow



MLflow Workflow Overview



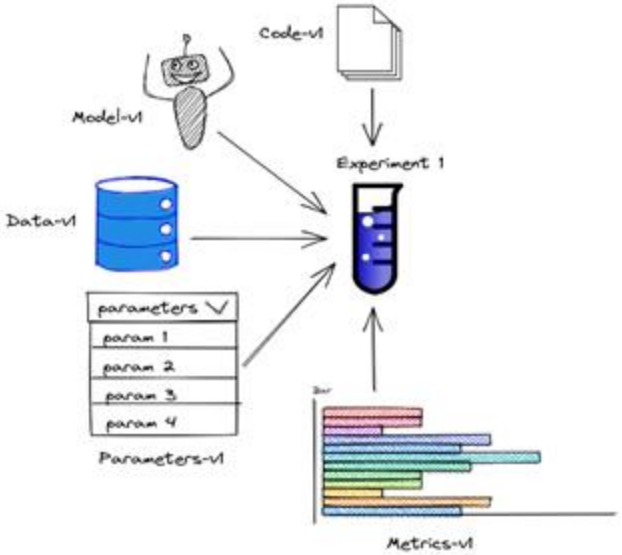
MLflow Experiment Tracking

Tracking Server

- Tracks **ML experiments** in MLflow
- **Centralized service** for storing logs
- Manages **parameters, metrics, artifacts,** and **metadata**

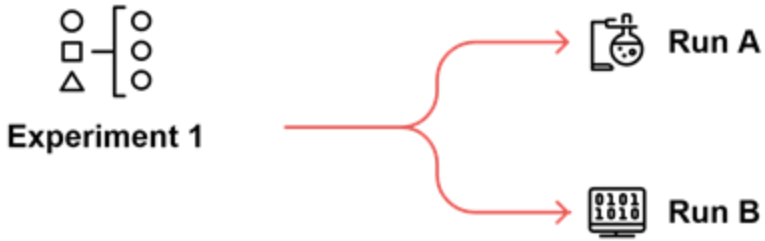
ML Experiment

- The process of **developing** a machine learning model
- Includes :
  - **data preparation**
  - **model training**
  - **evaluation**



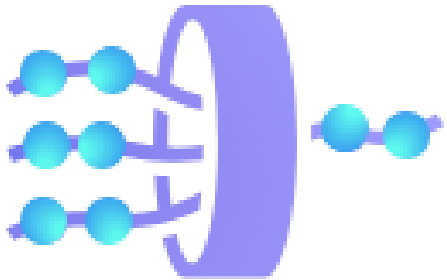
Experiment Run

- A **single iteration** or **trial** within an ML experiment
- A specific set of **parameters** and **configurations** are tested during a run



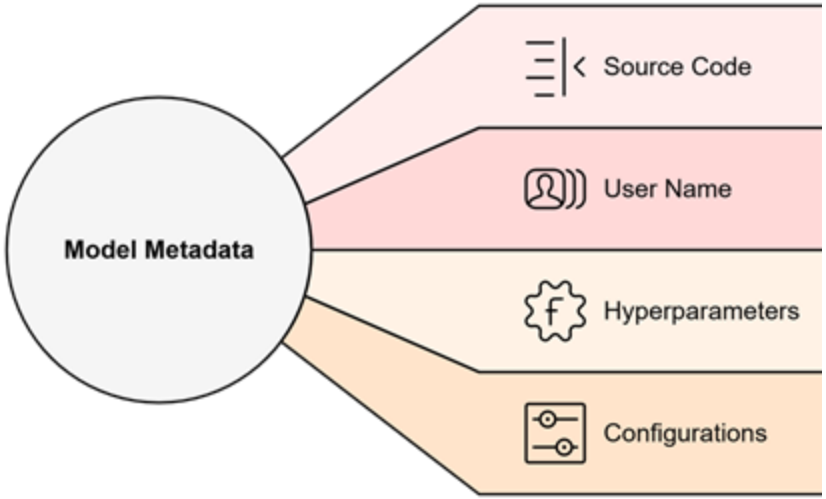
Run Artifact

- Any **file** or **output** generated during an experiment run
- Examples: model files, logs, metrics



Experiment Metadata

- Key **information** related to an experiment

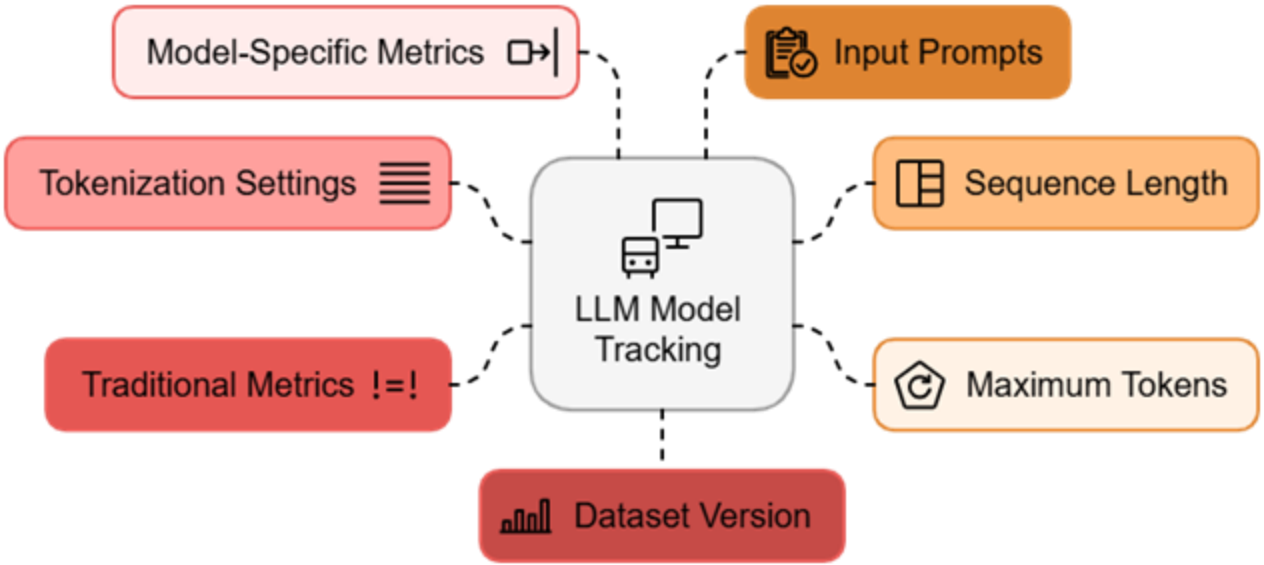


Experiment Tracking and LLMOps

LLMOps and Experiment Tracking

- Just like ML models, **LLMs** require **tracking**
- **Reproducibility** and **traceability** are essential for regulated industries using **LLMs**

What to track with LLMs



Summary

Key Takeaways

- Experiment tracking **optimizes** model development
- **MLflow, W&B, and Neptune AI** simplify ML tracking and organization
- **Clear versioning, documentation,** and **comparisons** ensure reliable models
- MLflow streamlines **ML workflows**
- Effective tracking of prompts, hyperparameters, datasets, and performance is crucial for LLMs

Pulse Check

Where does experiment tracking fit in your ML workflow?

Hands-on: MLflow for Air Quality Classification


O'REILLY®

O'REILLY

Introduction to Docker and ML Models Packaging

Ammar Mohanna

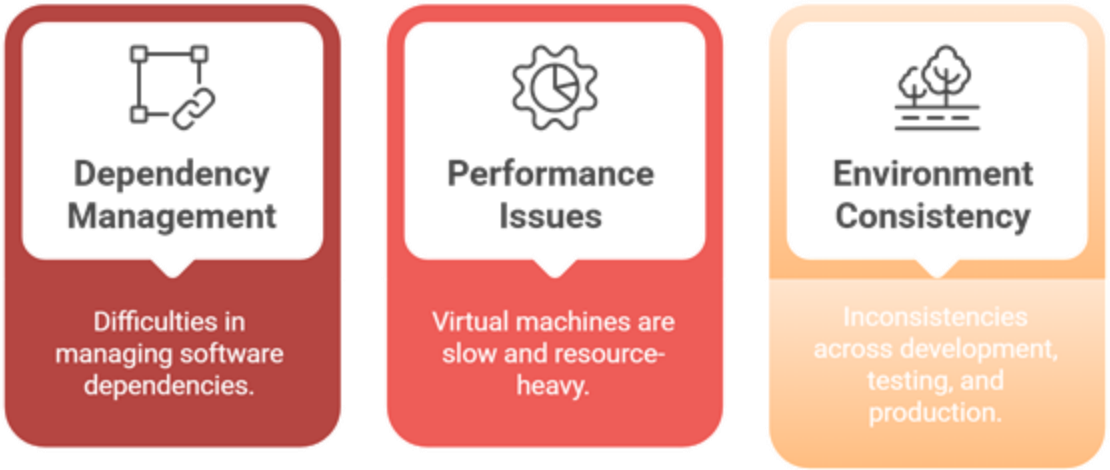
Learning Objectives

By the end of this module, you will be able to:

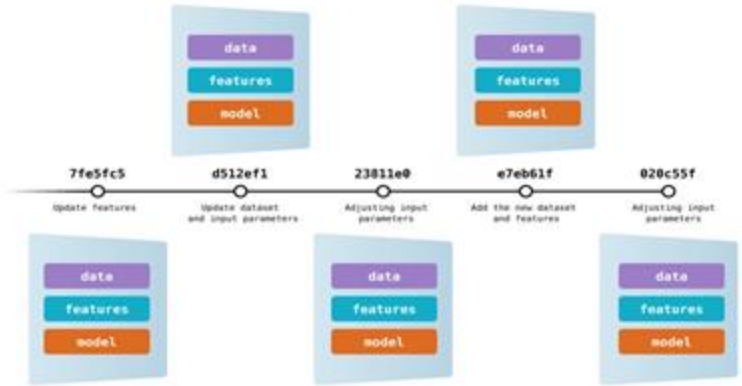
- Understand what is **containerization**
- Learn **containerization benefits** over VMs
- Understand Docker **architecture** and **components**
- Grasp key **Docker concepts**: images, Dockerfile, and containers

Containerization

Issues in Software Deployment



Version Control in ML

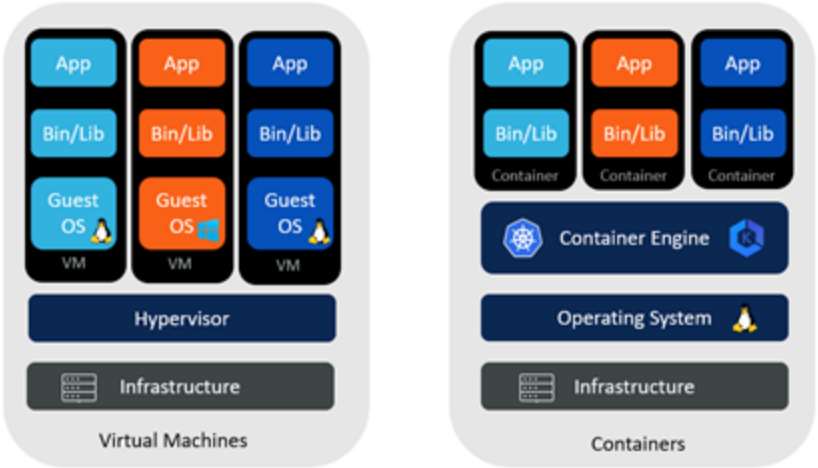


What is Containerization?

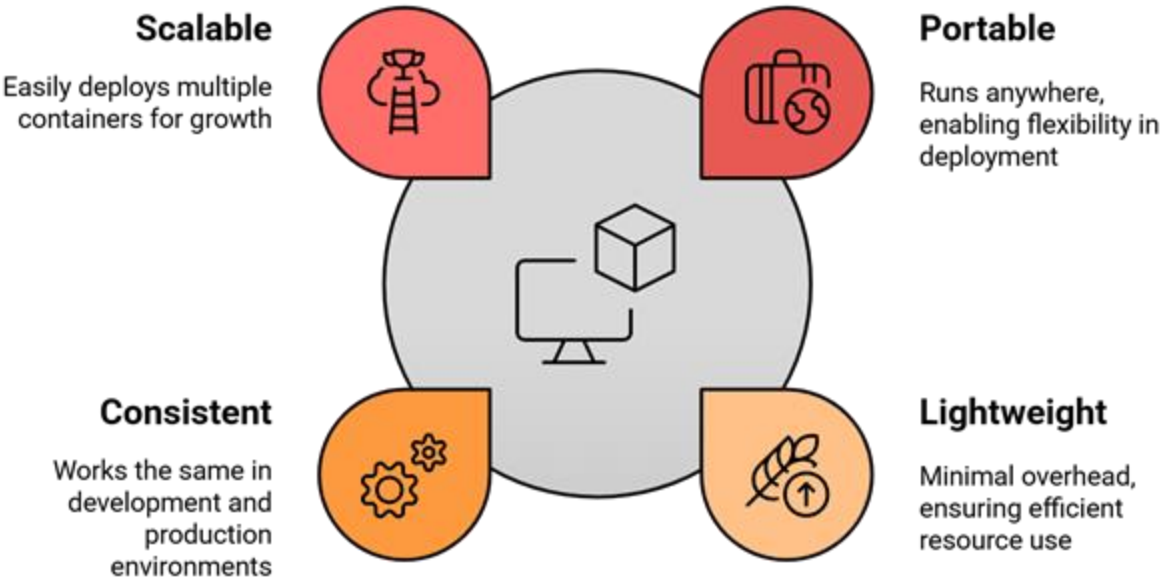
- Process of **packaging** software and its **dependencies** into a single, lightweight unit called a **container**

VM vs. Containers

- **VMs** provide well-separated runtime environments
- **Containers** offer efficient resource usage with smaller image sizes



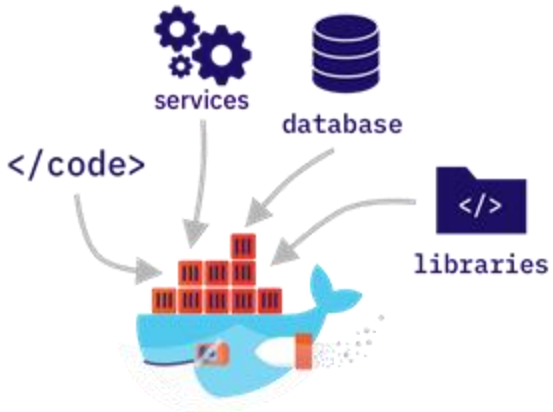
Why Containers



Docker

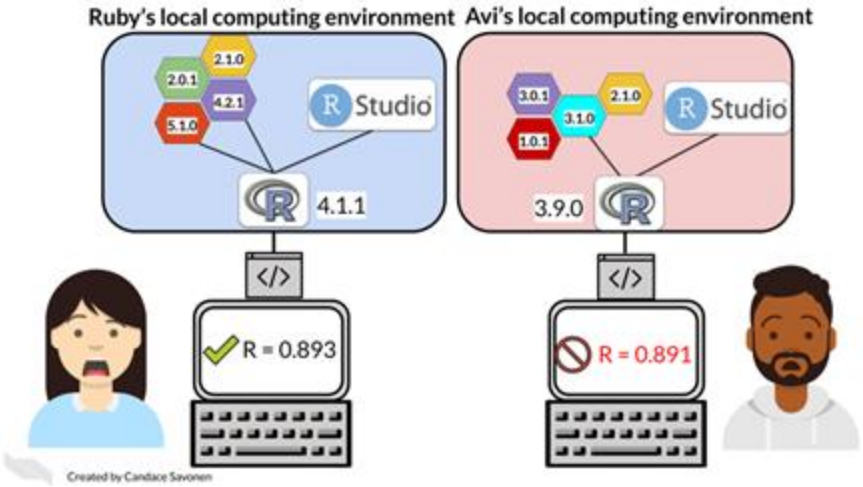
Docker

- **Definition:** Docker is an open-source platform for building, deploying, and managing containerized applications
- **Key Features:** Lightweight, portable, and efficient



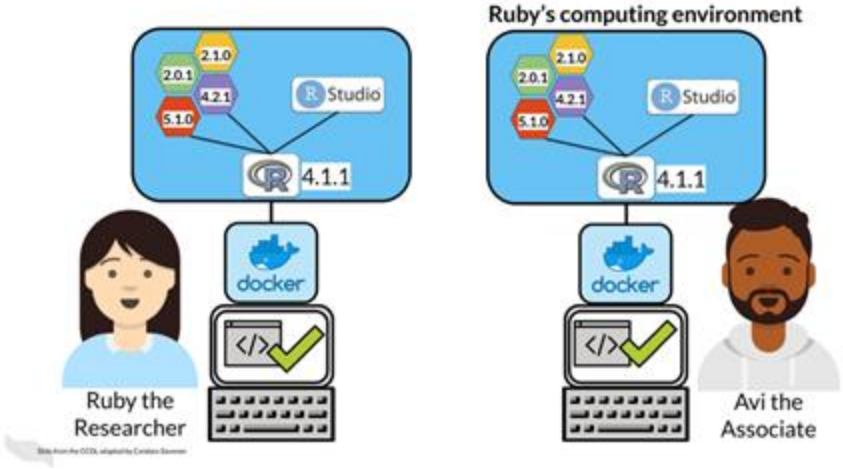
Docker for Reproducible Environment

- Before Docker, **different dependencies** often caused **inconsistent** results



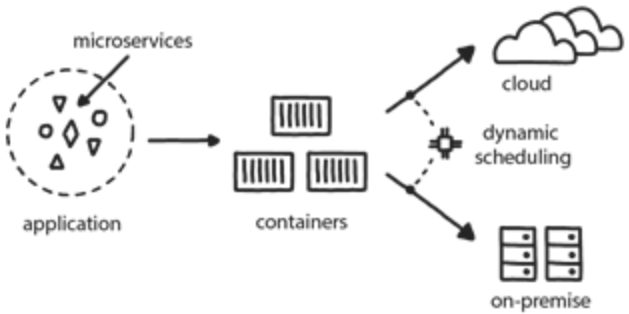
Docker for Reproducible Environment

- Docker solves this by **packaging** code and dependencies together, ensuring **consistency** across environments



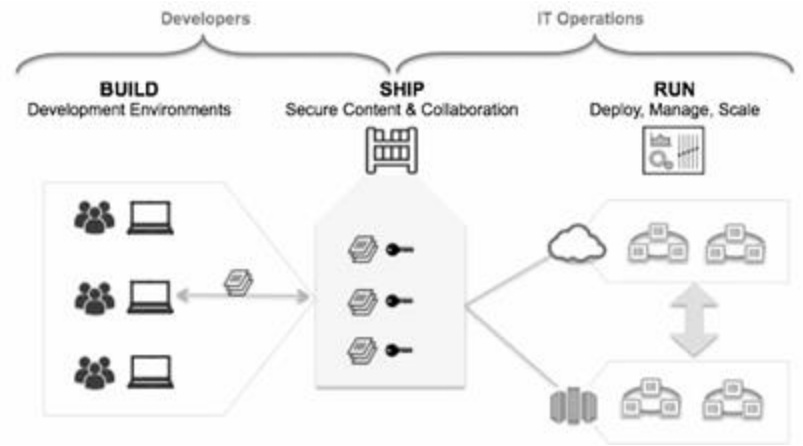
Support for Containment

- Docker acts as a **repository**, **client**, and **target** for deploying containerized applications



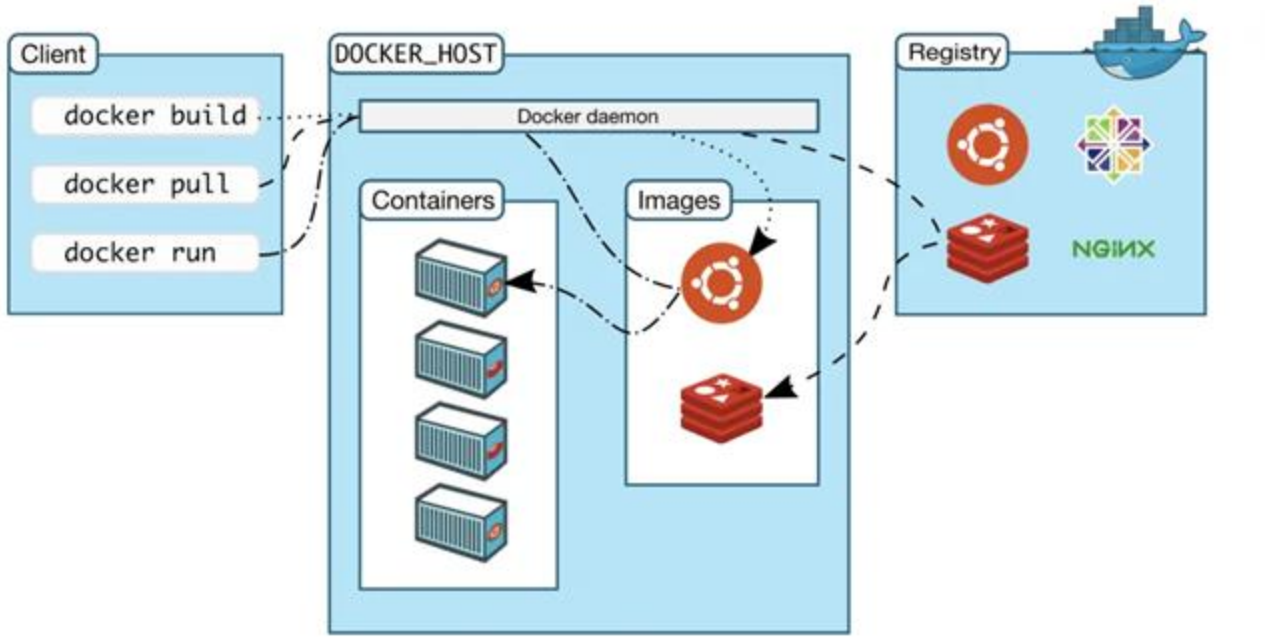
Separation of Concerns

- **Developers** focus on coding without worrying about the environment
- **Ops** handle deployment without worrying about the application code



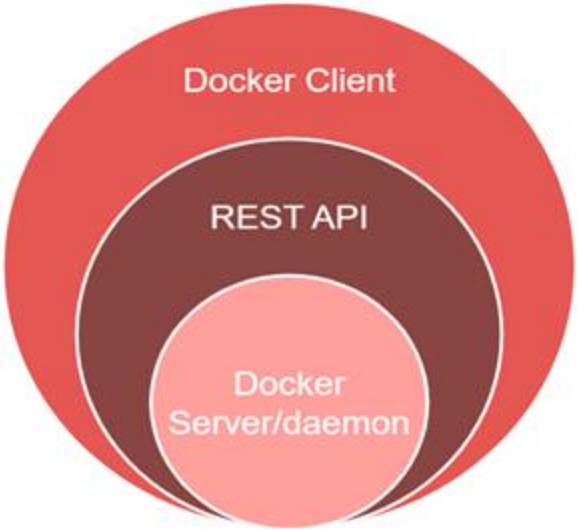
Docker Architecture

Docker Architecture



Docker Client

- Primary **user interface** for Docker
- Executing **commands**
- Communicates with the **Docker Daemon**



Docker Daemon

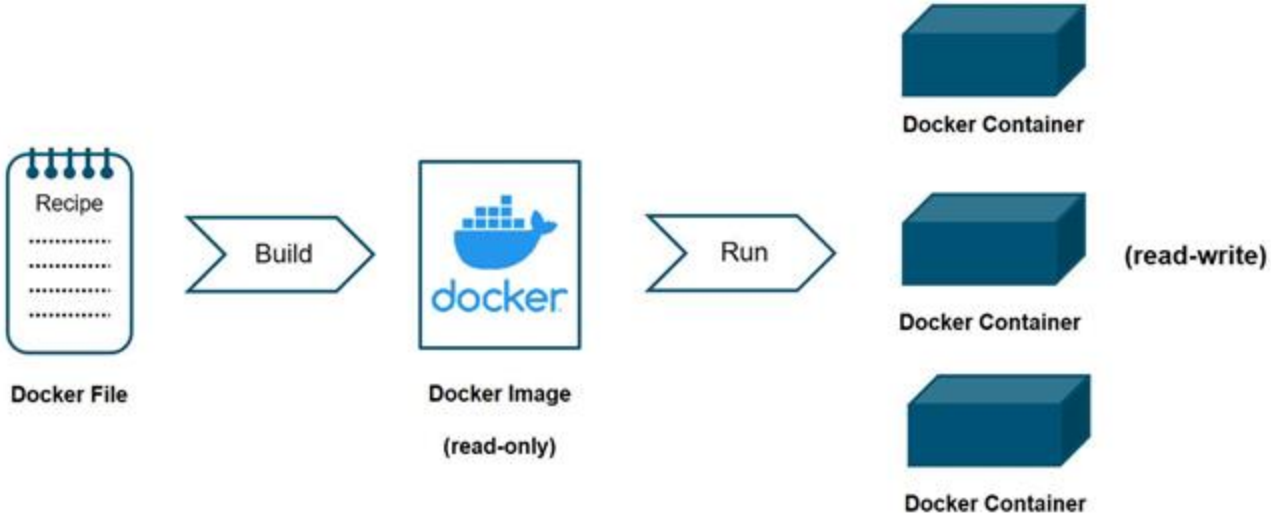
- Runs in the background on the **host** machine
- Responsible for Docker **containers**
- Handles **API requests**

Docker Registry

- Stores Docker **images**
- Can be **public** or **private**
- Allows image **sharing** and **distribution**

Key Concepts in Docker

Docker Lifecycle



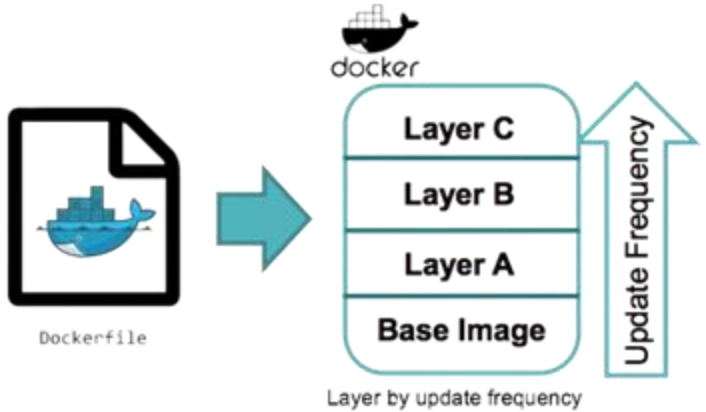
Dockerfile

- Script with **instructions** to build a Docker **image**
- Sets up the **environment**
- Defines the **base image** and application **configurations**

```
# 1. Base Image
FROM python:3.9-slim

# 2.Copy Files
COPY . /app

# 3.Install Dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 4.Run app.py
CMD ["python", "app.py"]
```

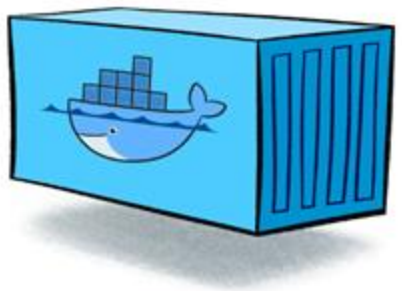
Images

- **Read-Only Templates**
- **Layered Architecture:** Each Dockerfile instruction creates a new layer
- **Reusability:** Ensures consistency across environments



Containers

- A running **instance** of a Docker image
- **Lightweight** and **isolated** environment
- Can be **started, stopped, moved,** and **deleted**



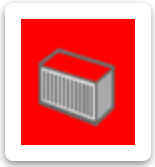
Docker Hub

- The default public **registry** for Docker images
- Offers **official** and **verified** images
- Provides **community-contributed** images

All Together: Workflow

Build

Develop an app in any language



Ship

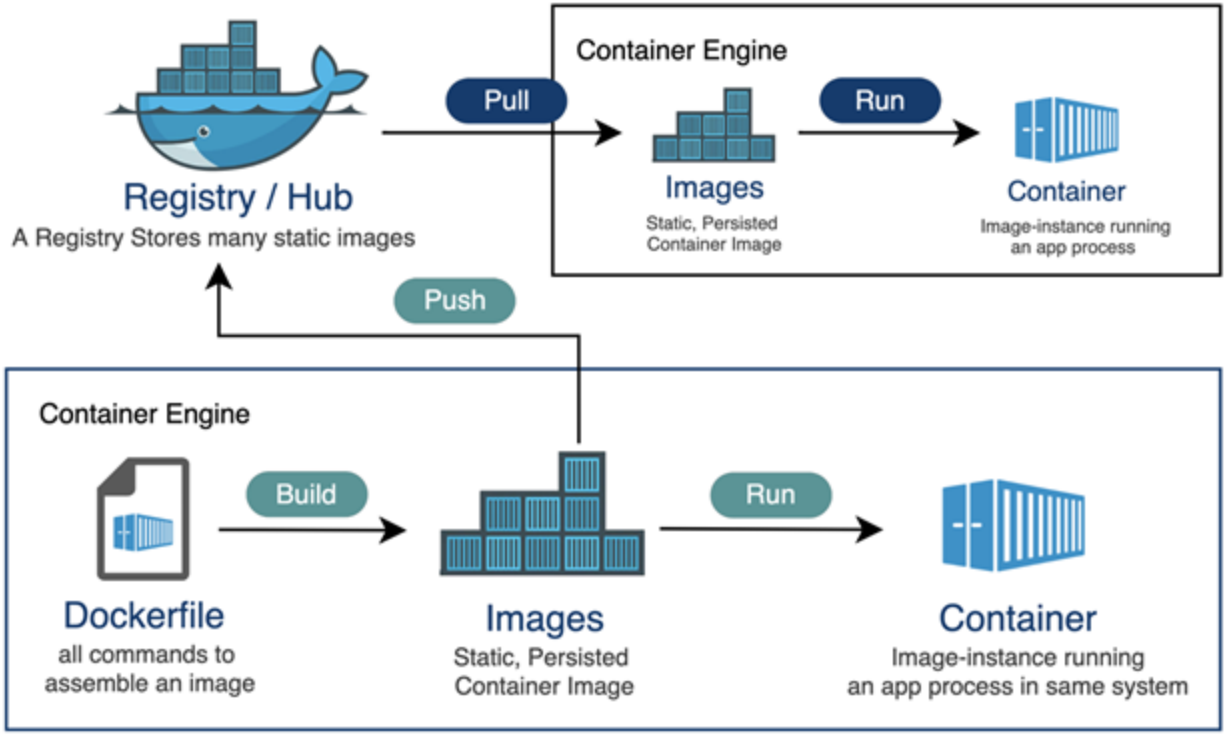
Ship the app anywhere or store them

Run

Scale to multiple nodes,deploy, and manage

Essential Docker Commands

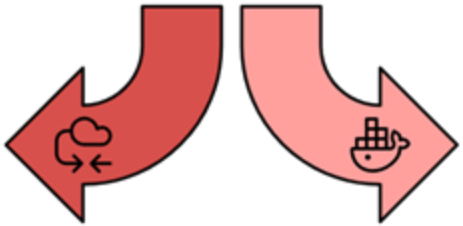
Essential Docker Commands



Setup and Images

Pull from Registry

Downloads an image from a registry like Docker Hub to the local machine.



Build from Dockerfile

Constructs a new image using instructions from a Dockerfile in the current directory.

`docker pull <image>`

`docker build -t <name> .`

Container Management

- **docker run <image>** : Creates and starts a container from the specified Docker image
- **docker stop <container>**: Stops a running container
- **docker ps -a**: Lists all containers, incl running, stopped, and exited ones.

Clean up

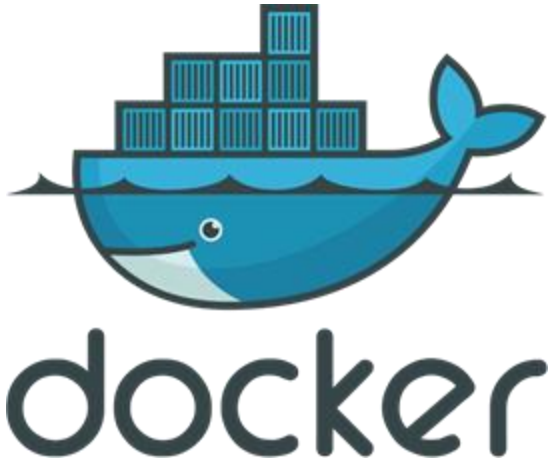
- **docker rm <container>:** Removes a stopped container
- **docker rmi <image>:** Deletes an unused Docker image

Hands-on: Deploying an Age Detection Model Using Docker

Summary

Key Takeaways

- **Containers** are lightweight and scalable
- Docker ensures **consistent and portable environments**
- Docker architecture includes key components such as the **Docker Client, Docker Daemon, and Docker Registry**
- **Image** is blueprint for containers
- **Dockerfile** is the script to build an image




O'REILLY®

Break

O'REILLY

CI/CD for Machine Learning

Ammar Mohanna

Learning Objectives

By the end of this lesson, you will be able to:

- Understand the **CI/CD pipeline** and its components
- Recognize how **CI/CD applies to Machine Learning**
- Implement automation to ensure **faster, safer, and more reliable ML deployments**
- Use **GitHub Actions** to automate testing and deployment of ML models

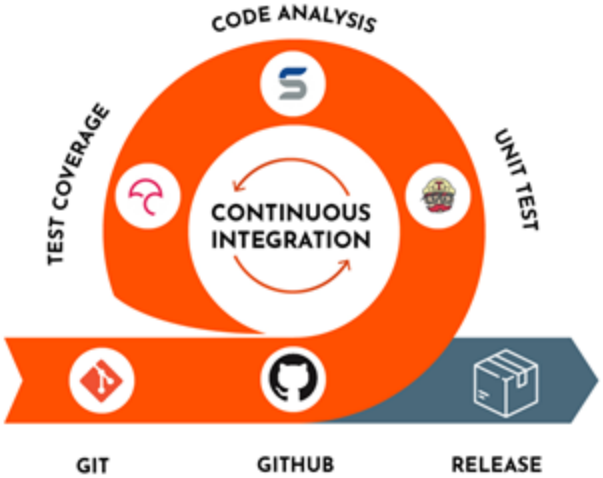
CI/CD

Continuous Integration and Continuous Deployment/Delivery (CI/CD)

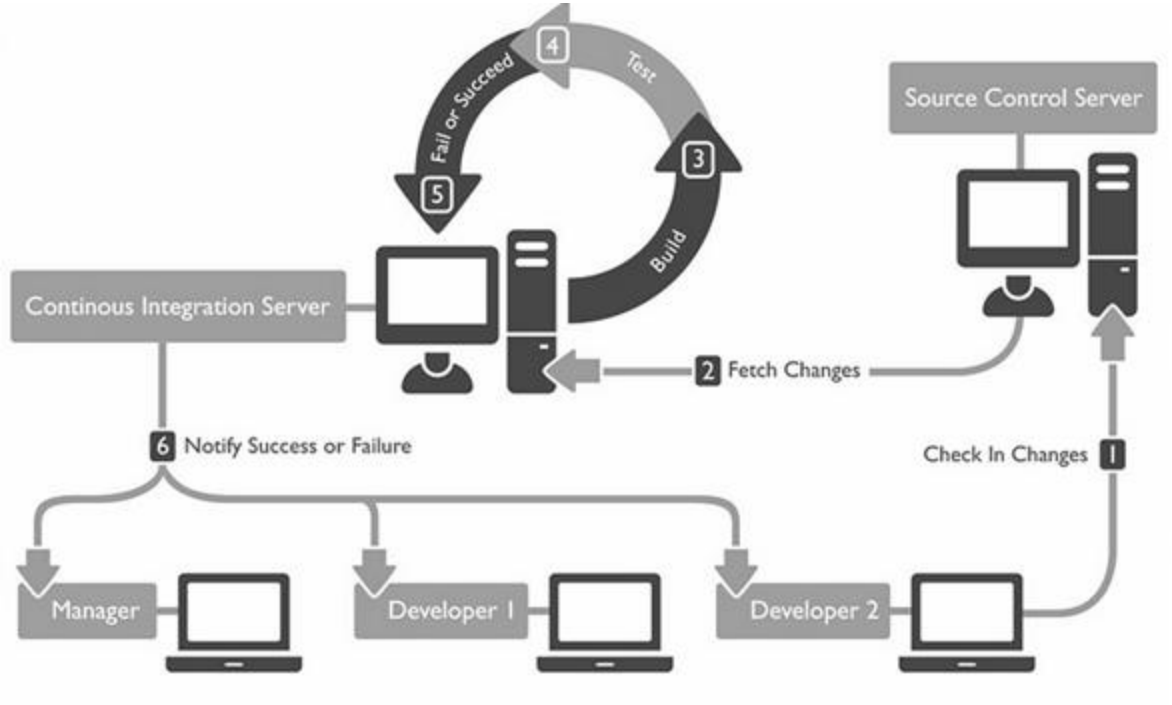
- CI/CD bridges the gap between **development** and **operations** by enforcing **automation** in:
  - Building
  - Testing
  - Deployment
- CI/CD enables faster, safer, and more reliable **software delivery**

Continuous Integration

- **Code, data,** and **configurations** are regularly merged and tested
- Errors are **caught early** through **automated tests**



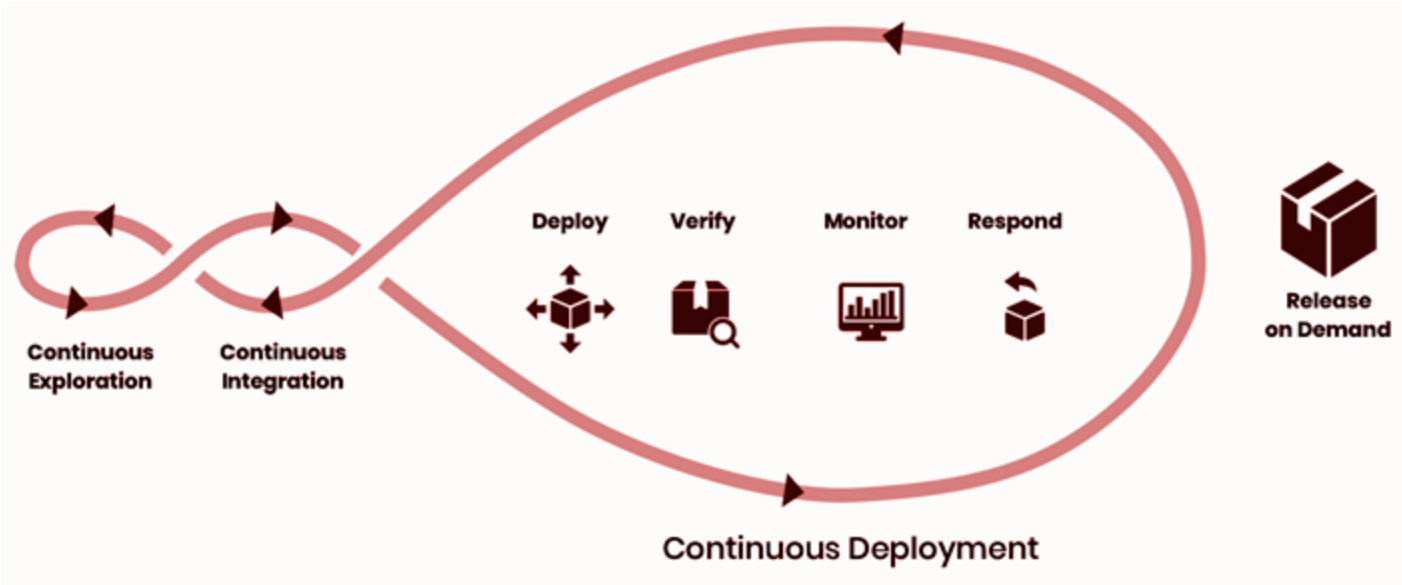
Continuous Integration Process



Continuous Deployment/Delivery

- Models are **automatically deployed** once they meet performance criteria
- Delivery of new versions is **seamless** and **repeatable**

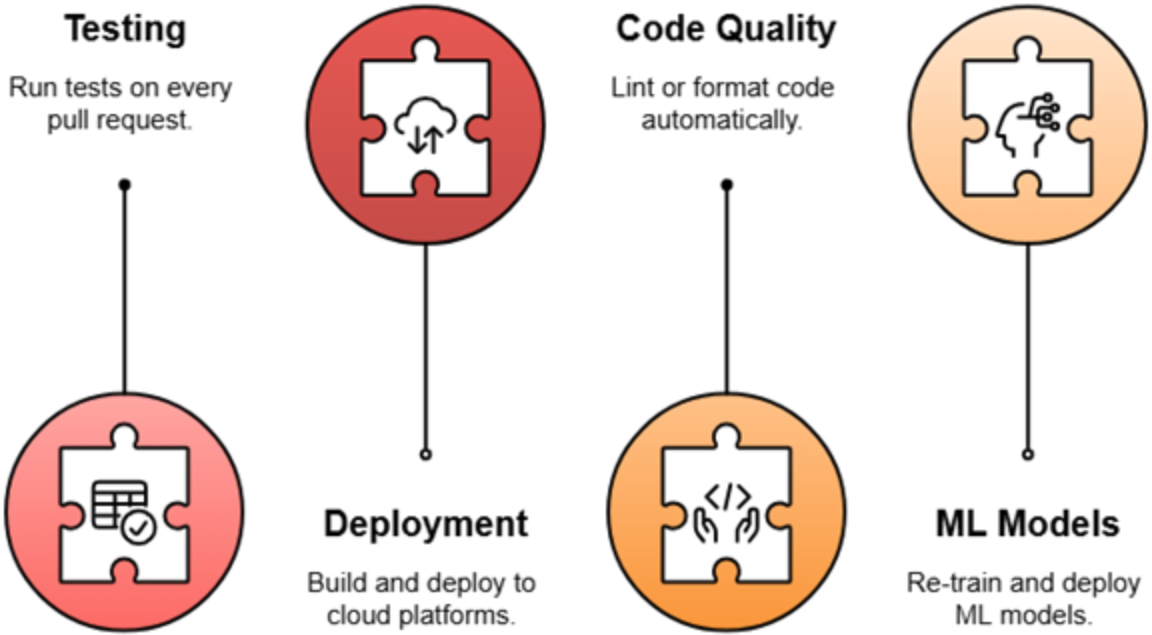
Continuous Deployment/Delivery Process



CI/CD vs. Traditional Deployment

Traditional Approach	CI/CD Approach
Large, infrequent updates	Small, frequent changes
Manual testing and deployment	Automated tests and deployment
High-risk, big-bang releases	Safer, incremental rollouts

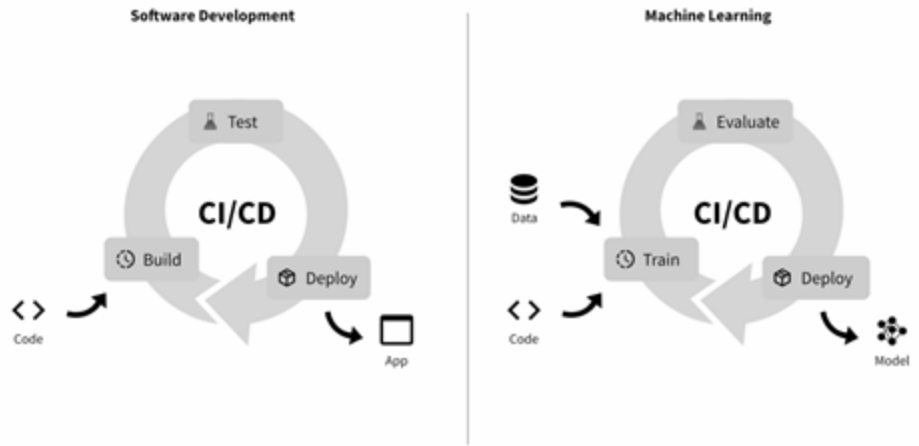
Use Cases:


Testing
Run tests on every pull request.
Deployment
Build and deploy to cloud platforms.
Code Quality
Lint or format code automatically.
ML Models
Re-train and deploy ML models.

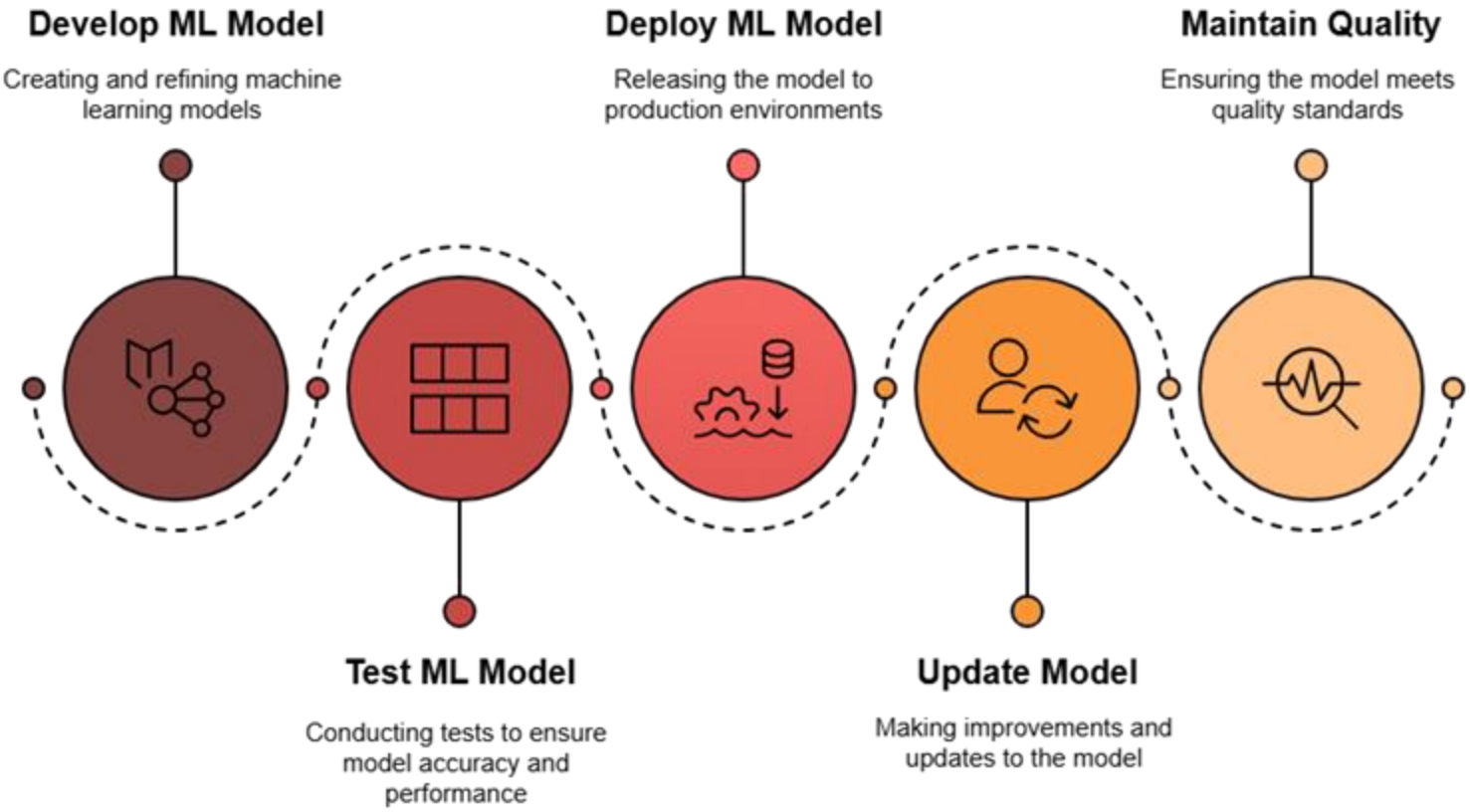
CI/CD for ML

CI/CD for Machine Learning

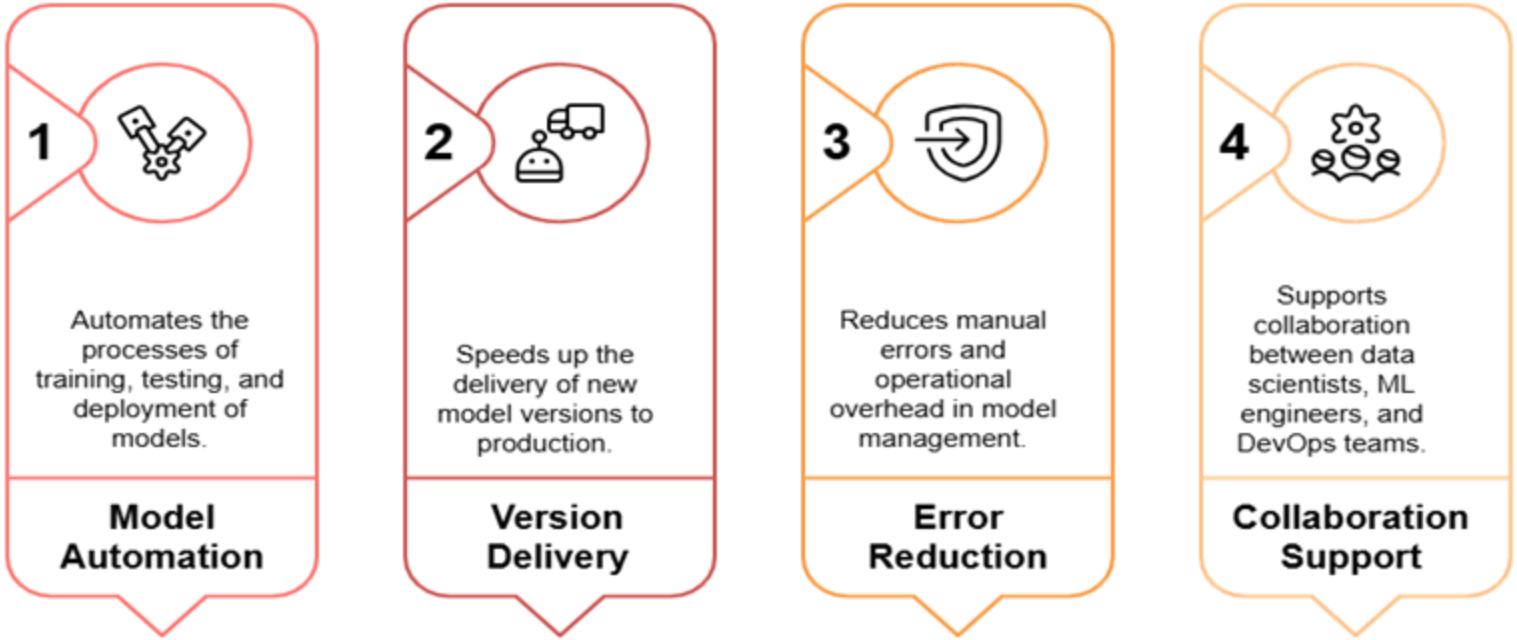
- ML is more than code
- Pipeline includes **data validation, training, evaluation, deployment**
- Needs **performance-based checks** (e.g., accuracy)
- Deploys based on **model quality**



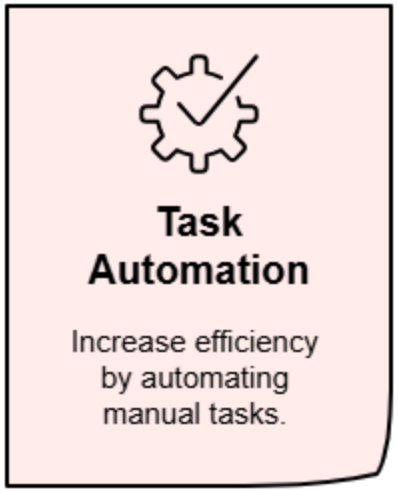
CI/CD for Machine Learning Process



Why CI/CD for ML

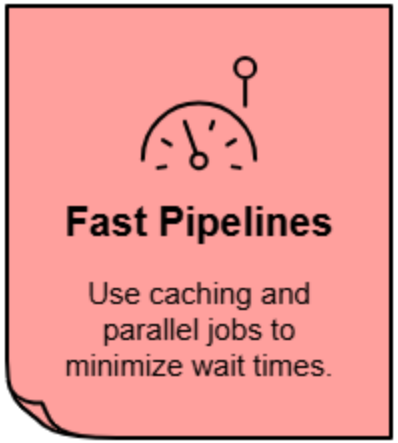


Best Practices



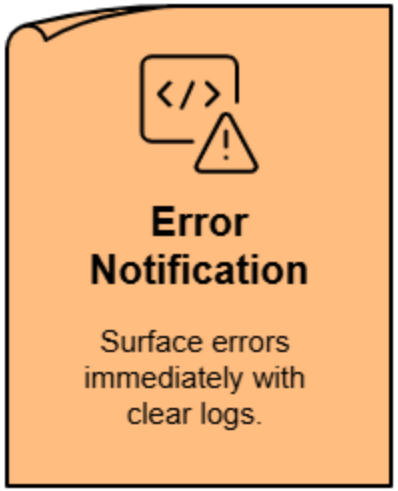
Task Automation

Increase efficiency by automating manual tasks.



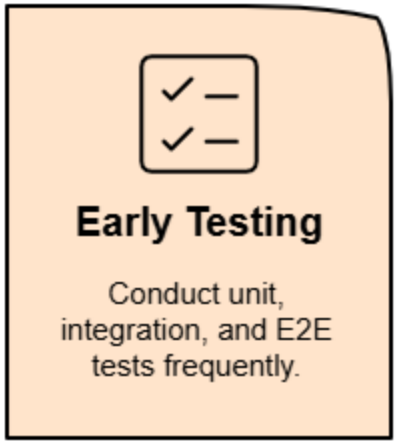
Fast Pipelines

Use caching and parallel jobs to minimize wait times.



Error Notification

Surface errors immediately with clear logs.



Early Testing

Conduct unit, integration, and E2E tests frequently.

GitHub Actions

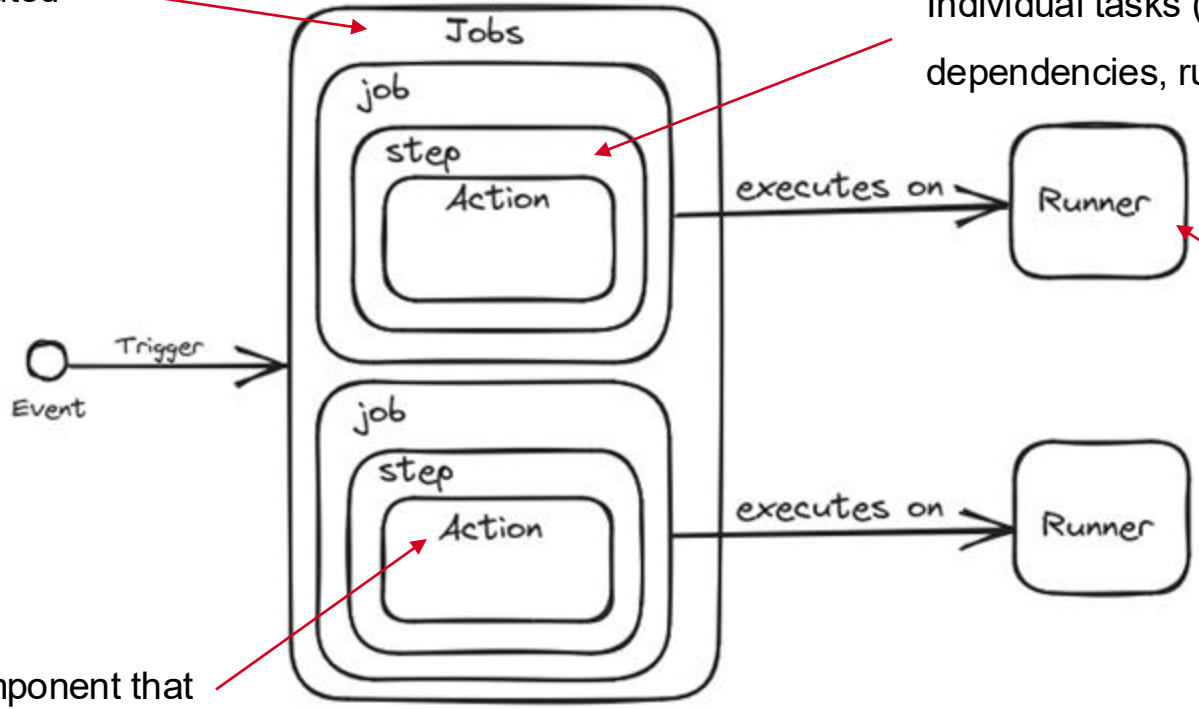
What is GitHub Actions?

- **GitHub Actions** is a built-in automation tool
- Automates **CI/CD workflows** in your GitHub repo
- Supports tasks like **build, test,** and **deploy**
- Streamlines the ML lifecycle

Key Concepts

A set of steps executed on a runner

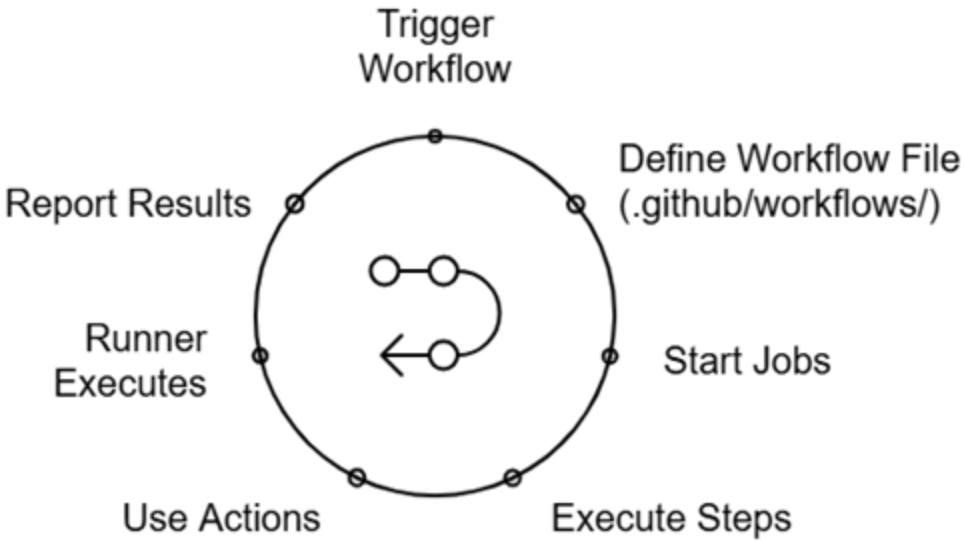
Individual tasks (e.g., install dependencies, run tests)



Machine that runs the workflow

Reusable component that performs a specific task

GitHub Actions Workflow Process



Summary

Key Takeaways

- **CI/CD** automates building, testing, and deploying ML workflows
- **CI** catches issues early with frequent code merges and tests
- **CD** automates deployment once models meet performance criteria
- In ML, it also covers data validation, training, and evaluation
- Tools like **GitHub Actions** simplify and automate the entire pipeline

Hands-on: Using GitHub Actions to test and deploy a model

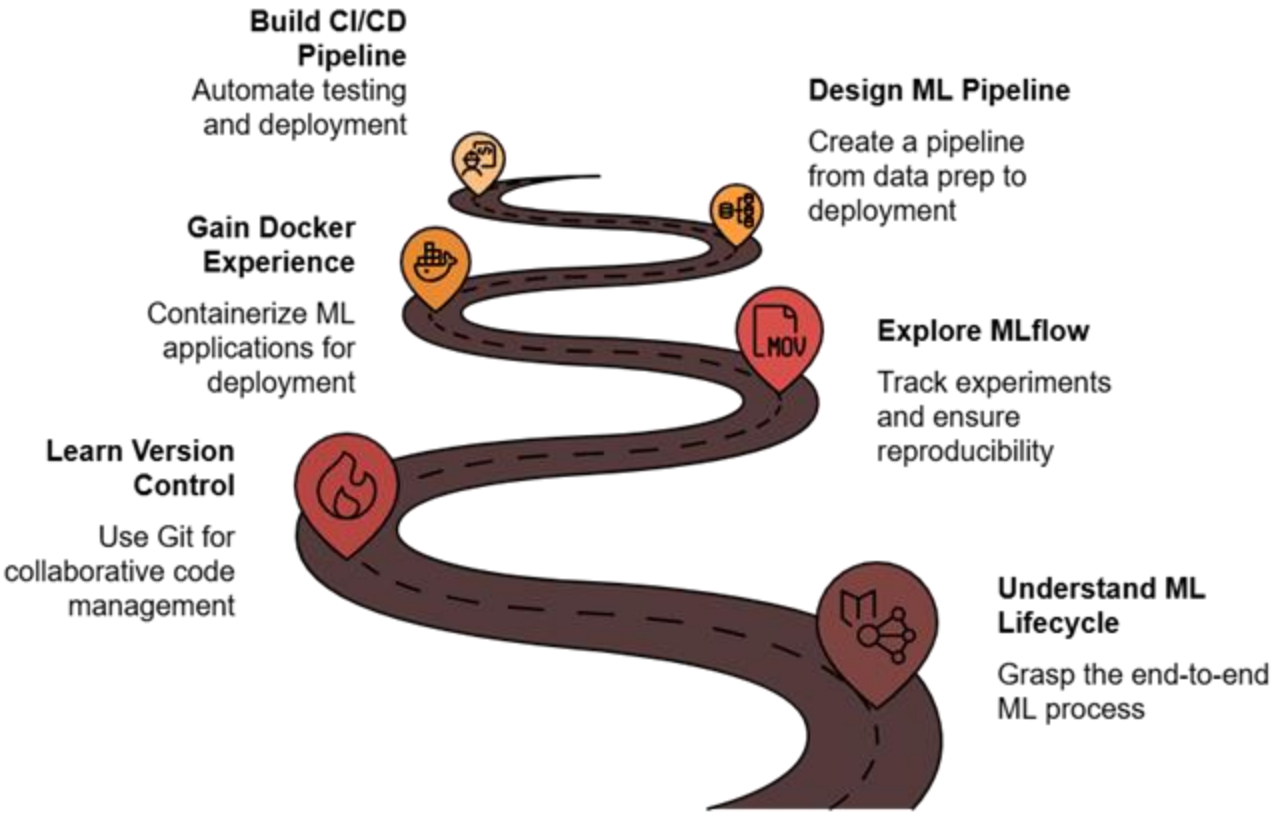

O'REILLY®

O'REILLY

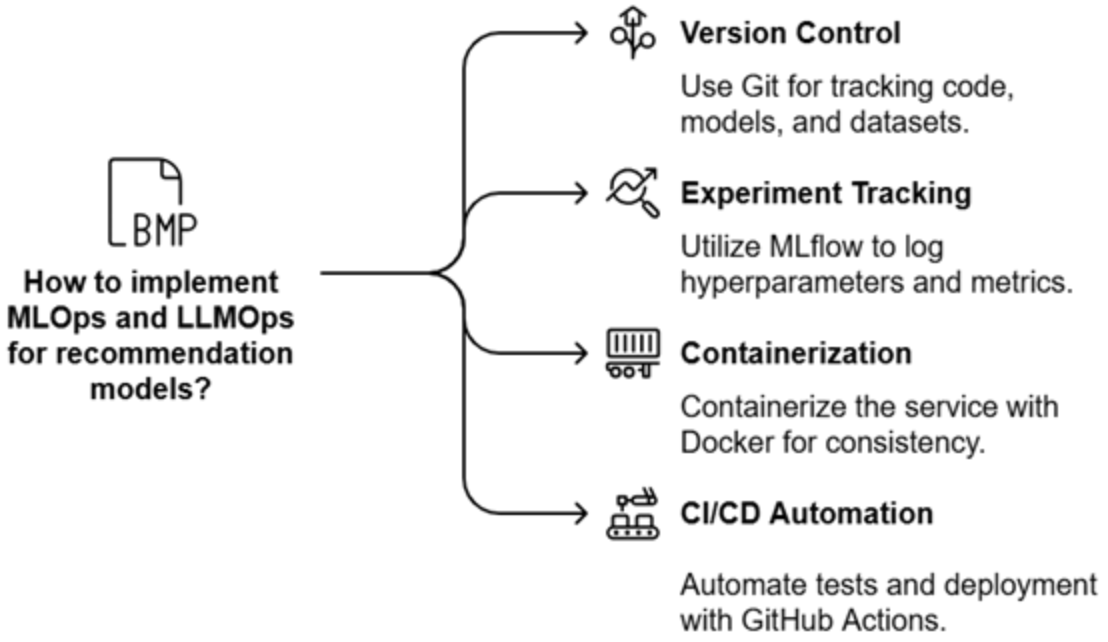
Recap

Ammar Mohanna

Key Learnings



All Together: ML Workflow



Day 2

LLMOps Spotlight #1

Learn LLM integration in pipelines

Scaling ML Systems

Strategies for high-throughput inference

Advanced Kubernetes for ML Deployments

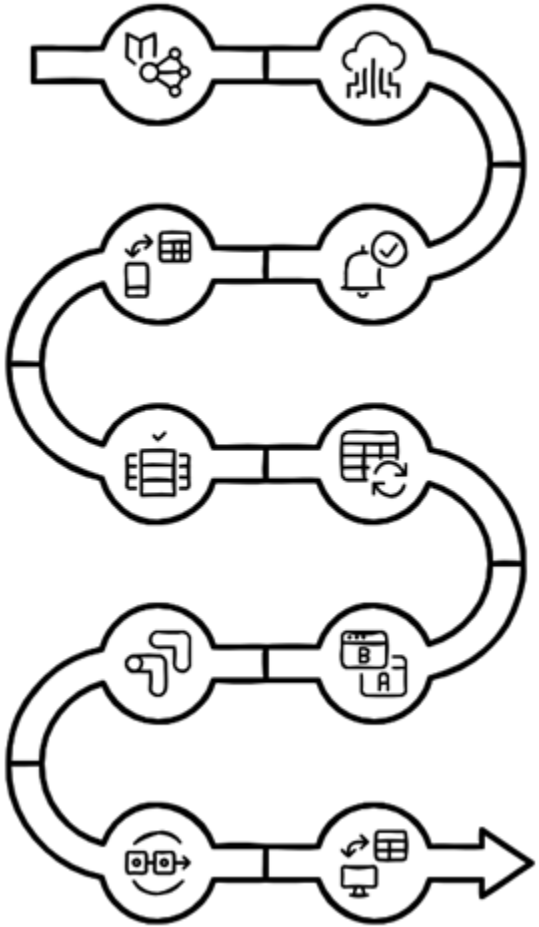
Auto-scaling and high availability

LLMOps Spotlight #2

Operational challenges of LLMs

Model Lifecycle Management & Governance

Track and manage models



Orchestrating ML Workloads with Kubernetes

Scale and deploy ML services

Model Monitoring and Logging

Track performance and set up alerts

ML Workflow Orchestration

Automate complex workflows

Advanced Experimentation

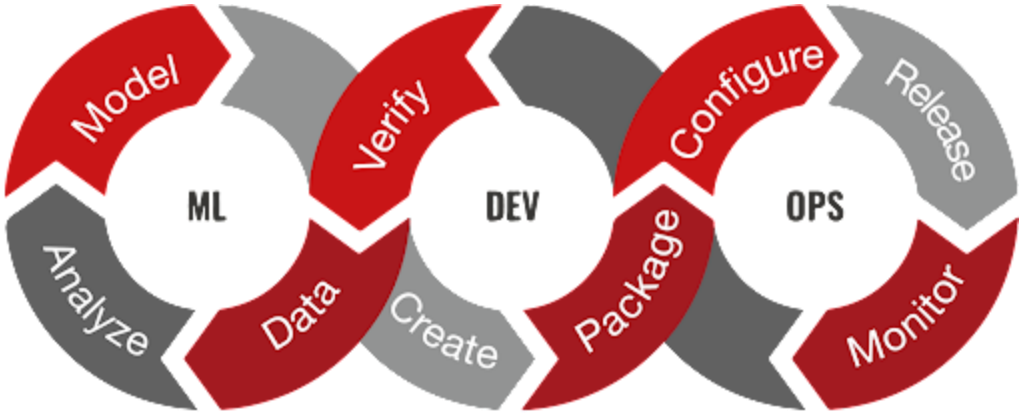
A/B testing and model validation

Capstone Demo

End-to-end MLOps integration

Conclusion

- Equipped with tools to streamline ML workflows
- Ensured reproducibility and collaboration across teams
- Automated model deployment and testing with CI/CD



Thank you

Q & A


O'REILLY®